

MoveInsight: Herramienta para el análisis de patrones de movimiento y prevención de lesiones

Anexo II. Análisis y diseño del sistema

Trabajo de Fin de Grado

Grado en Ingeniería Informática



**VNiVERSiDAD
D SALAMANCA**

Junio de 2026

Autor

Diego Gómez Terradillos

Tutora

Dña. Carolina Zato Domínguez

Índice de Contenidos

Índice de Contenidos	iii
Índice de Tablas.....	iv
Índice de Figuras	v
1 Introducción	1
1.1 Introducción al modelo de análisis	1
1.2 Introducción al modelo de diseño.....	2
2 Modelo de dominio	3
3 Realización de los casos de uso	5
3.1 Catálogo de clases de análisis.....	5
4 Diagramas de secuencia.....	8
5 Diagrama de clases y paquetes de análisis.....	13
6 Arquitectura del sistema	14
6.1 Estilo arquitectónico	14
6.2 Modelo C4	15
6.3 Decisiones arquitectónicas	17
7 Diseño de la funcionalidad.....	21
7.1 Clases de diseño	21
7.2 Diagramas de secuencia de diseño	25
7.3 Diagramas de estados y de actividad.....	27
7.4 Patrones de diseño	30
8 Diseño de interfaces.....	31
8.1 Interfaz de usuario	31
8.2 Interfaces externas de la API	32
9 Diseño de datos	34
9.1 Modelo lógico.....	34
9.2 Implementación de la base de datos	35
10 Diagrama de despliegue	37
Glosario de términos.....	39

Índice de Tablas

Tabla 1. Clases de interfaz.....	5
Tabla 2. Clases de control	6
Tabla 3. Clases de entidad	6
Tabla 4. Clases participantes por caso de uso	7
Tabla 5. ADR-01	18
Tabla 6. ADR-02	18
Tabla 7. ADR-03	18
Tabla 8. ADR-04	19
Tabla 9. ADR-05	19
Tabla 10. ADR-06	19
Tabla 11. Matriz de decisiones x NFR	20
Tabla 12. Clases de análisis y su implementación en el diseño	22
Tabla 13. Clases del diseño por capas	25
Tabla 14. Patrones de diseño	30
Tabla 15. Interfaces API: Autenticación	32
Tabla 16. Interfaces API: Sesiones.....	32
Tabla 17. Interfaces API: Analítica	33
Tabla 18. Interfaces API: Check-in de dolor.....	33
Tabla 19. Interfaces API: Incidencias	33
Tabla 20. Interfaces API: Bienestar.....	33
Tabla 21. Interfaces API: Recomendaciones.....	33

Índice de Figuras

Figura 1. Modelo de dominio	4
Figura 2. Diagrama de secuencia: registro y verificación de cuenta	8
Figura 3. Diagrama de secuencia: subir vídeo y análisis.....	9
Figura 4. Diagrama de secuencia: consultar el informe de una sesión.....	9
Figura 5. Diagrama de secuencia: consultar readiness	10
Figura 6. Diagrama de secuencia: consultar recomendaciones	10
Figura 7. Diagrama de secuencia: iniciar sesión	11
Figura 8. Diagrama de secuencia: consultar bienestar	11
Figura 9. Diagrama de secuencia: consultar análisis.....	12
Figura 10. Diagrama de secuencia: exportar informe PDF	12
Figura 11. Diagrama de paquetes de análisis	13
Figura 12. Diagrama C4: Nivel de contexto.....	16
Figura 13. Diagrama C4: Nivel de contenedores	16
Figura 14. Diagrama C4: Nivel de componentes	17
Figura 15. Diagrama de clases de diseño	23
Figura 16. Diagrama de clases de infraestructura	24
Figura 17. Diagrama de diseño: iniciar sesión	26
Figura 18. Diagrama de diseño: subir vídeo y análisis asíncrono	26
Figura 19. Diagrama de diseño: consultar el informe con sondeo	27
Figura 20. Diagrama de estados: ciclo de vida de una sesión	27
Figura 21. Diagrama de actividad: canalización de análisis del vídeo.....	28
Figura 22. Diagrama de actividad: cálculo del readiness	29
Figura 23. Pantallas de la interfaz	31
Figura 24. Modelo lógico (BD).....	34
Figura 25. Diagrama de despliegue	37

1 Introducción

Este anexo da continuidad al Anexo I (Especificaciones del sistema): este establecía qué debe hacer MoveInsight, el presente describe cómo se estructura el sistema para satisfacer esos requisitos. Su contenido se organiza en dos grandes bloques. El primero, de análisis, construye un modelo del sistema a partir de los casos de uso y los requisitos de información ya definidos, sin comprometerse todavía con decisiones tecnológicas concretas. El segundo, de diseño, toma ese modelo como punto de partida para concretar la arquitectura, la funcionalidad, las interfaces, el modelo de datos y el despliegue sobre la plataforma seleccionada.

Para la elaboración de este anexo se ha usado un conjunto reducido de herramientas. La redacción y el maquetado del documento se han realizado con Microsoft Word, que ha permitido estructurar el contenido mediante estilos, tablas normalizadas, índices automáticos y referencias cruzadas. El diagrama de casos de uso se ha generado con PlantUML, una herramienta que produce diagramas UML a partir de una descripción textual en lugar de dibujarlos manualmente; este enfoque facilita mantener un estilo homogéneo, rehacer diagramas con rapidez ante cualquier cambio en los casos de uso.

Las referencias citadas en este anexo, en formato [Autor, año], se recogen en la bibliografía de la memoria principal.

1.1 Introducción al modelo de análisis

El modelo de análisis constituye una primera aproximación a la estructura interna del sistema, expresada en un nivel de abstracción independiente de la tecnología de implementación. Su finalidad es comprender y organizar el problema en los conceptos del dominio, las responsabilidades y las interacciones entre ellos, antes de afrontar las decisiones propias del diseño. Actúa, por tanto, como puente entre la especificación de requisitos del Anexo I y el modelo de diseño que se desarrolla más adelante en este mismo documento.

Para estructurar el modelo, MoveInsight emplea los estereotipos de clases de análisis propios de UML, que clasifican las clases en tres tipos. Las clases interfaz representan la interacción entre los actores y el sistema, como las pantallas de la aplicación móvil o los puntos de entrada de la API. Las clases de control coordinan la lógica de cada caso de uso, como el análisis del vídeo o el cálculo del readiness. Por último, las clases entidad

modelan los conceptos del dominio, como el usuario, la sesión, el análisis biomecánico o las recomendaciones. Esta separación de responsabilidades permite razonar sobre el comportamiento del sistema sin atarlo aún a un marco tecnológico determinado

A lo largo de este bloque de análisis se desarrollan, sucesivamente, el modelo de dominio y su glosario de clases, que fija el vocabulario conceptual del sistema; la realización de los casos de uso definidos en el Anexo I mediante las clases de análisis; la vista de interacción, con los diagramas de secuencia de los casos de uso más representativos; el diagrama de clases de análisis, que integra la estructura resultante; y el diagrama de paquetes, que agrupa esas clases en unidades cohesionadas. En conjunto, ofrecen una visión del sistema suficientemente completa para abordar con garantías la fase de diseño.

1.2 Introducción al modelo de diseño

El bloque de diseño parte de ese modelo de análisis para transformarlo en una solución concreta, ya condicionada por las decisiones tecnológicas y por los atributos de calidad recogidos en los requisitos no funcionales del Anexo I. En primer lugar se describe la arquitectura del sistema: el estilo arquitectónico adoptado, una estructura cliente-servidor en la que la aplicación móvil consume los servicios de un backend que concentra el análisis biomecánico, junto con su representación mediante el modelo C4 [Brown, 2018] (contexto, contenedores y componentes) y las principales decisiones arquitectónicas que la justifican.

A continuación se aborda el diseño de la funcionalidad, con las clases de diseño, sus diagramas de interacción y los patrones de diseño [Gamma et al., 1995] aplicados; el diseño de las interfaces, tanto la de usuario, con sus bocetos y el mapa de navegación de la aplicación, como las interfaces de programación expuestas por la API; el diseño de datos, que concreta el modelo lógico, la estrategia de persistencia y el modelo físico de la base de datos; y, por último, el diagrama de despliegue, que sitúa los artefactos del sistema sobre la infraestructura real. El resultado es un modelo de diseño suficientemente detallado para guiar la implementación del sistema.

2 Modelo de dominio

El modelo de dominio representa los conceptos fundamentales del problema que MoveInsight resuelve y las relaciones entre ellos, en un nivel de abstracción independiente de la implementación.

No se trata de un modelo de datos ni anticipa cómo se almacenará o procesará la información, sino de un modelo conceptual que recoge los conceptos centrales del dominio necesarios para entender el problema, derivados directamente de los casos de uso del Anexo I.

En MoveInsight, el modelo gira en torno al Usuario (el deportista y su perfil) y a la sesión de entrenamiento, que se graba en un Vídeo y se descompone en las Repeticiones analizadas, cada una con su valoración biomecánica (ResultadoRepetición), de las que se obtiene el Informe de la sesión. Alrededor del deportista se modelan también su estado de forma (Readiness), las Incidencias que sufre, los registros de dolor posteriores a cada sesión (CheckInDolor) y las Recomendaciones que recibe. Cada concepto se describe con sus atributos y sus relaciones, sin presuponer su realización técnica.

Quedan fuera del modelo las salidas puramente analíticas o de presentación, como la validación retrospectiva o los insights, que no son conceptos del dominio sino resultados que el sistema elabora a partir de ellos.

El modelo de dominio fija además un vocabulario común y preciso que debe mantenerse coherente a lo largo del análisis y del diseño. La responsabilidad de cada clase, es decir, el concepto que representa dentro del problema, se detalla en el catálogo de clases de entidad del apartado siguiente.

En la figura 1 se detalla el modelo de dominio:

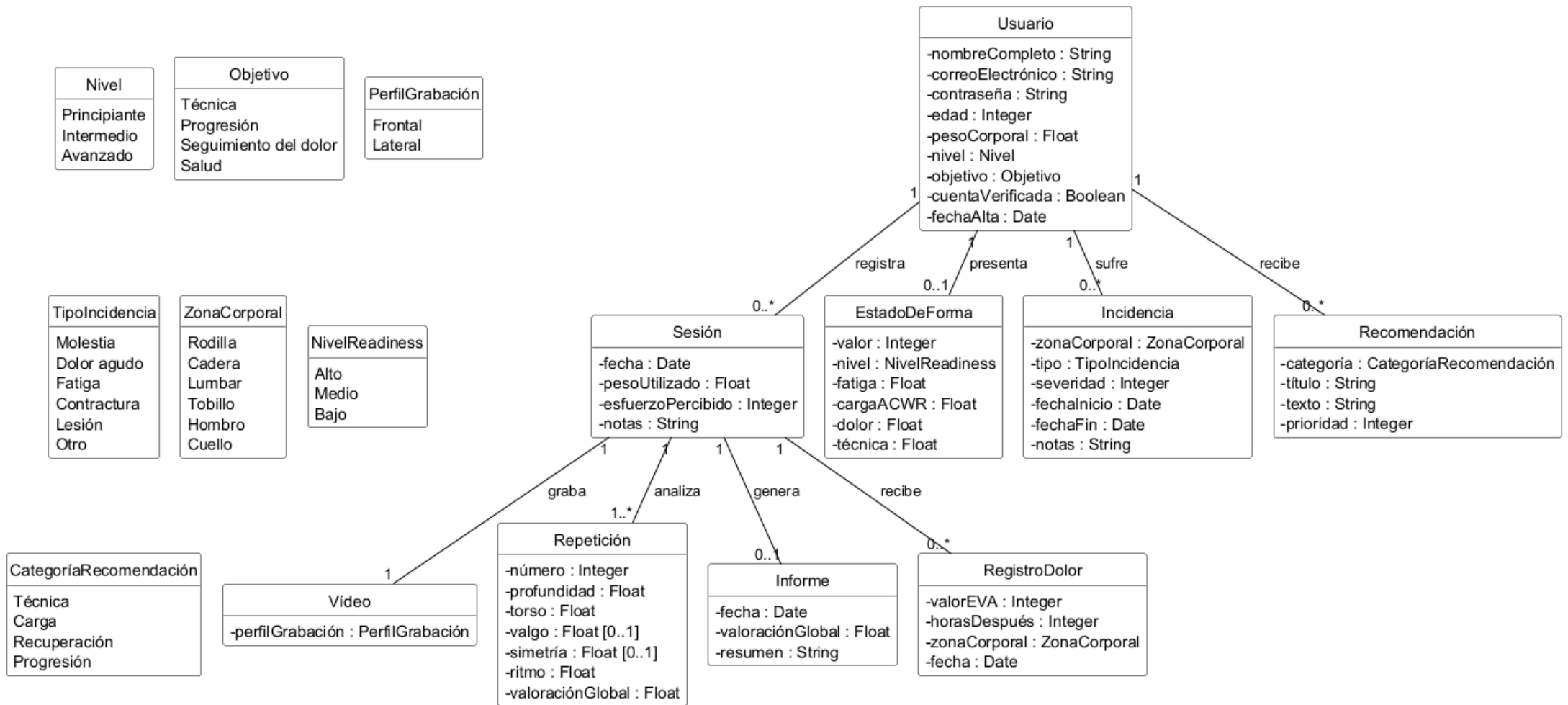


Figura 1. Modelo de dominio

3 Realización de los casos de uso

Este apartado describe cómo se llevan a cabo los casos de uso definidos en el Anexo I mediante la colaboración de un conjunto de clases de análisis. Siguiendo los estereotipos de UML, estas clases se clasifican en tres tipos: las clases de interfaz, que representan la interacción del actor con el sistema; las clases de control, que coordinan la lógica de cada caso de uso; y las clases de entidad, que son los conceptos del modelo de dominio sobre los que se opera.

3.1 Catálogo de clases de análisis

A continuación, se presenta el catálogo de clases de análisis identificadas y, después, la realización de los casos de uso: una tabla que asocia cada caso de uso con las clases que lo materializan. Las clases identificadas se presentan agrupadas por su estereotipo (interfaz, control y entidad) en las tres tablas siguientes.

Clase interfaz	Caso(s) de uso
PantallaLogin	CU-03
PantallaRegistro	CU-01
PantallaVerificaciónCorreo	CU-02
PantallaRecuperarContraseña	CU-05
PantallaCambiarContraseña	CU-06
PantallaOnboarding	CU-01
PantallaPrincipal	CU-04, CU-09, CU-13
PantallaPanel	CU-14, CU-15, CU-16
PantallaCaptura	CU-07
PantallaResultados	CU-08
PantallaVÍdeoEsqueleto	CU-08
PantallaReadiness	CU-10
PantallaBienestar	CU-12
PantallaFactoresIncidencia	CU-12
PantallaCheckInDolor	CU-12

Tabla 1. Clases de interfaz

Nota: las consultas de historial (CU-09), recomendaciones (CU-13), gráficos (CU-14) e insights (CU-15) se concentran en las pantallas Principal y Panel. Las notificaciones (CU-17, CU-18) se entregan mediante notificaciones push, sin pantalla dedicada.

Clase control	Responsabilidad	Caso(s) de uso
ControlAutenticación	Registro, verificación, inicio/cierre de sesión y gestión de contraseñas	CU-01 a CU-06
ControlSesión	Alta de la sesión, subida del vídeo, historial y consulta del informe	CU-07, CU-08, CU-09
ServicioAnálisisVídeo	Procesamiento del vídeo: estimación de pose, conteo de repeticiones y cálculo de KPIs	CU-11
ServicioReadiness	Cálculo del readiness, el ACWR y la fatiga	CU-10, CU-12, CU-13
ControlBienestar	Línea de bienestar, incidencias, check-ins de dolor y análisis de factores	CU-12
ControlRecomendaciones	Evaluación de disparadores y selección de las recomendaciones aplicables	CU-13
ControlAnalítica	Resumen estadístico, récords, gráficas de evolución y exportación	CU-14, CU-15, CU-16

Tabla 2. Clases de control

Clase entidad	Descripción
Usuario	Deportista registrado; mantiene su perfil (peso, nivel, objetivo) y sus credenciales.
Sesión	Entrenamiento registrado a partir de un vídeo de sentadilla, con su peso y esfuerzo percibido.
Vídeo	Grabación de la serie analizada, con su perfil (frontal o lateral).
ResultadoRepetición	Valoración biomecánica de una repetición (profundidad, torso, valgo, simetría, ritmo).
Informe	Síntesis de los resultados de una sesión que se presenta al deportista.
Incidencia	Molestia o lesión registrada por el deportista, con zona, tipo y duración.
CheckInDolor	Registro de dolor (EVA) posterior a una sesión, asociado a una zona corporal.
Readiness	Estado de forma del deportista a partir de su carga, fatiga, dolor y técnica.
Recomendación	Consejo que el sistema ofrece al deportista según su estado.

Tabla 3. Clases de entidad

3.1.1 Realización de los casos de uso

Una vez identificado el catálogo de clases de análisis, este apartado muestra cómo colaboran para llevar a cabo los casos de uso del sistema. La tabla siguiente asocia cada caso de uso con las clases interfaz, control y entidad que participan en su realización.

Caso de uso	Interfaz	Control	Entidad
CU-01 Registrarse	PantallaRegistro	ControlAutenticación	Usuario
CU-02 Confirmar cuenta	PantallaVerificaciónCorreo	ControlAutenticación	Usuario
CU-03 Iniciar sesión	PantallaLogin	ControlAutenticación	Usuario
CU-04 Cerrar sesión	PantallaPrincipal	ControlAutenticación	Usuario
CU-05 Recuperar contraseña	PantallaRecuperarContraseña	ControlAutenticación	Usuario
CU-06 Cambiar contraseña	PantallaCambiarContraseña	ControlAutenticación	Usuario
CU-07 Subir o grabar vídeo	PantallaCaptura	ControlSesión	Sesión
CU-08 Consultar informe	PantallaResultados, PantallaVídeoEsqueleto	ControlSesión	Sesión, ResultadoRepetición
CU-09 Consultar historial	PantallaPrincipal	ControlSesión	Sesión
CU-10 Consultar readiness	PantallaReadiness	ServicioReadiness	Sesión, ResultadoRepetición, CheckInDolor, Readiness
CU-11 Procesar vídeo	(actor Sistema)	ServicioAnálisisVídeo, ServicioReadiness	Sesión, ResultadoRepetición
CU-12 Consultar bienestar	PantallaBienestar, PantallaFactoresIncidencia, PantallaCheckInDolor	ControlBienestar, ServicioReadiness	Incidencia, CheckInDolor, Sesión
CU-13 Consultar recomendaciones	PantallaPrincipal	ControlRecomendaciones, ServicioReadiness	Sesión, ResultadoRepetición, CheckInDolor, Recomendación
CU-14 Consultar gráficos	PantallaPanel	ControlAnalítica	Sesión, ResultadoRepetición
CU-15 Consultar análisis	PantallaPanel	ControlAnalítica	Sesión, ResultadoRepetición
CU-16 Exportar PDF	PantallaPanel	ControlAnalítica	Sesión, ResultadoRepetición, Readiness, Recomendación

Tabla 4. Clases participantes por caso de uso

Nota: CU-17 y CU-18 (notificaciones) se resuelven mediante el servicio de notificaciones *push* y no participan clases de análisis del servidor.

4 Diagramas de secuencia

Una vez identificadas las clases de análisis y la forma en que colaboran, este apartado presenta la vista de interacción del sistema mediante diagramas de secuencia. Cada diagrama describe, el orden temporal en que los objetos se intercambian mensajes para producir el comportamiento esperado.

No se modelan los dieciocho casos de uso, sino una selección de los más representativos. El detalle de implementación por capas se reserva para los diagramas de secuencia de diseño.

Figura 2: Registro y verificación de cuenta (CU-01 + CU-02). El deportista se registra; el control de autenticación crea el usuario como cuenta no verificada y solicita la verificación del correo. En una segunda fase, el deportista confirma la cuenta y el control la marca como verificada, habilitando el acceso.

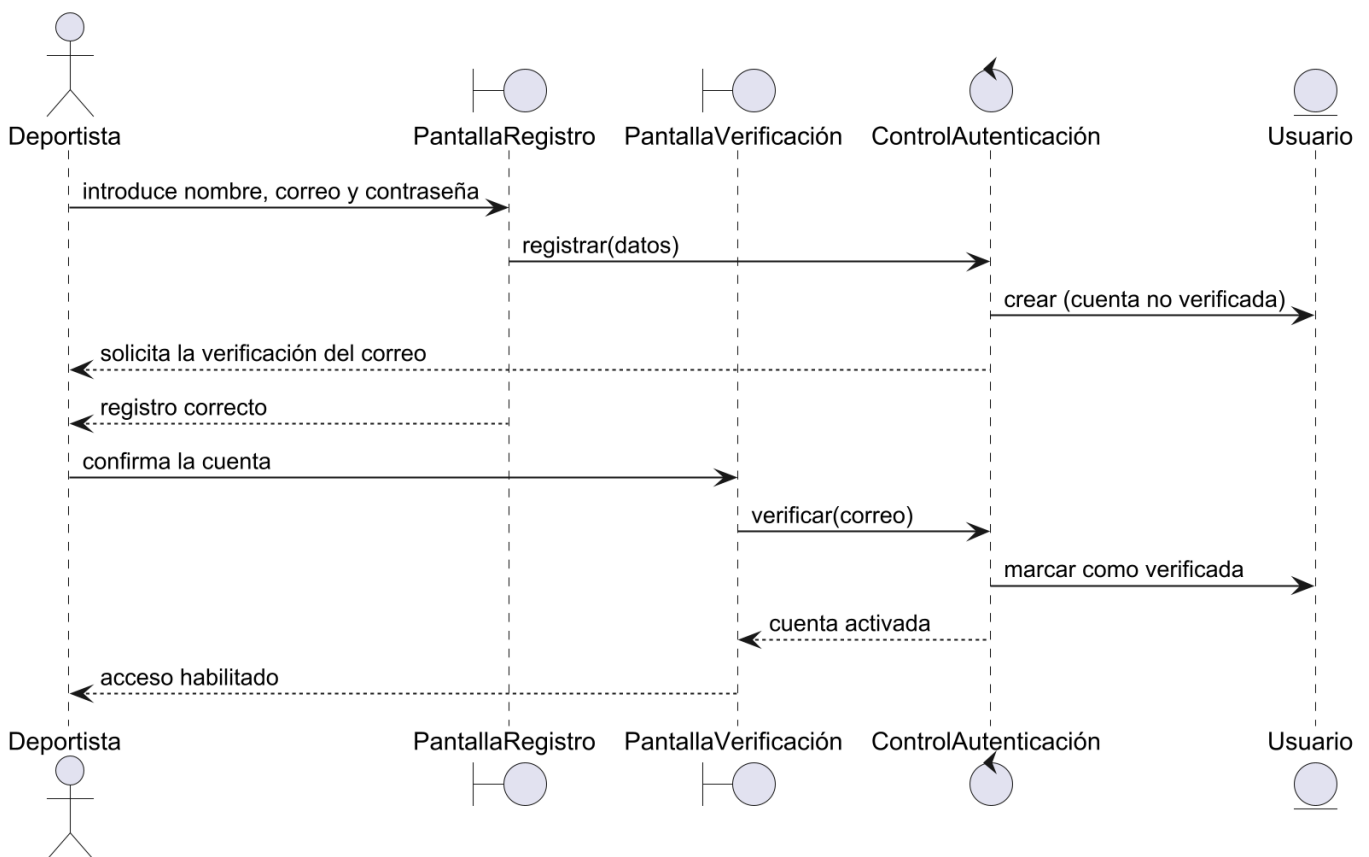


Figura 2. Diagrama de secuencia: registro y verificación de cuenta

Figura 3: Subir vídeo y análisis (CU-07 + CU-11). El núcleo del sistema. La pantalla de captura entrega el vídeo al control de sesión, que crea la sesión y registra el vídeo. El servicio de análisis procesa entonces el vídeo —estimación de pose, conteo de repeticiones y cálculo de los KPIs—, genera el resultado de cada repetición y el informe, y recalcula el estado de forma del deportista.

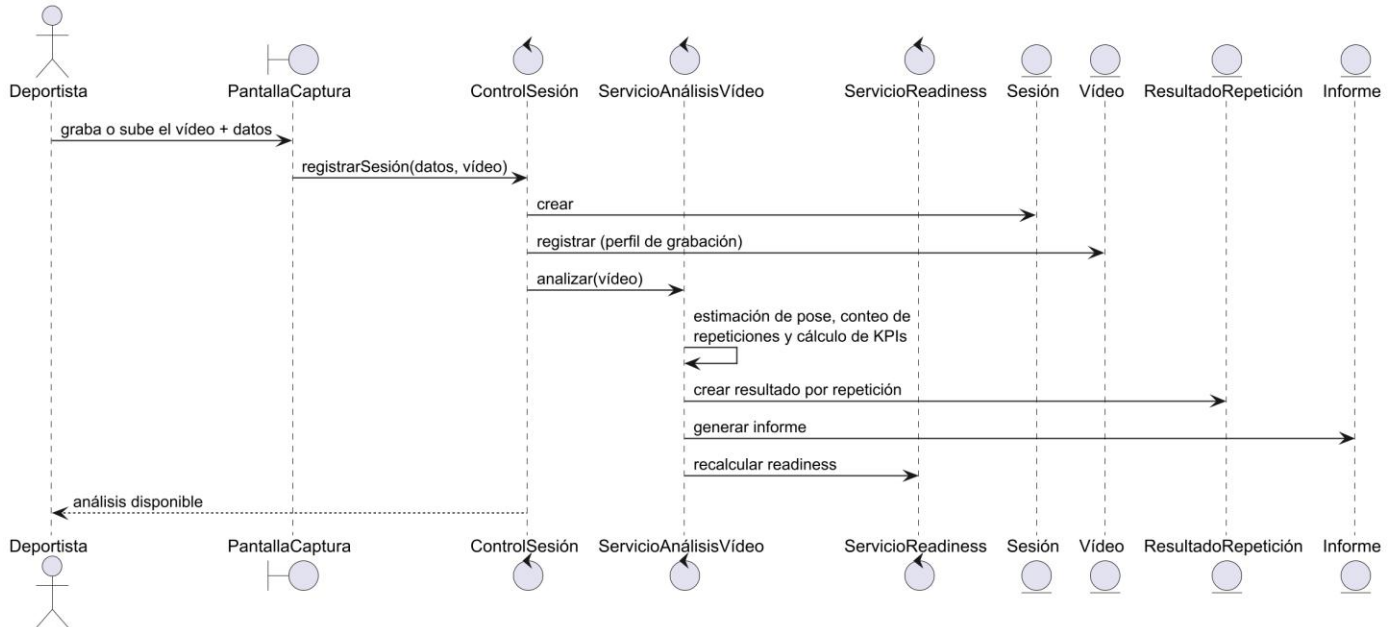


Figura 3. Diagrama de secuencia: subir vídeo y análisis

Figura 4: Consultar el informe de una sesión (CU-08). El deportista abre la sesión, la pantalla solicita sus resultados al control de sesión y este devuelve los KPIs por repetición y el vídeo con el esqueleto, que componen el informe que se muestra.

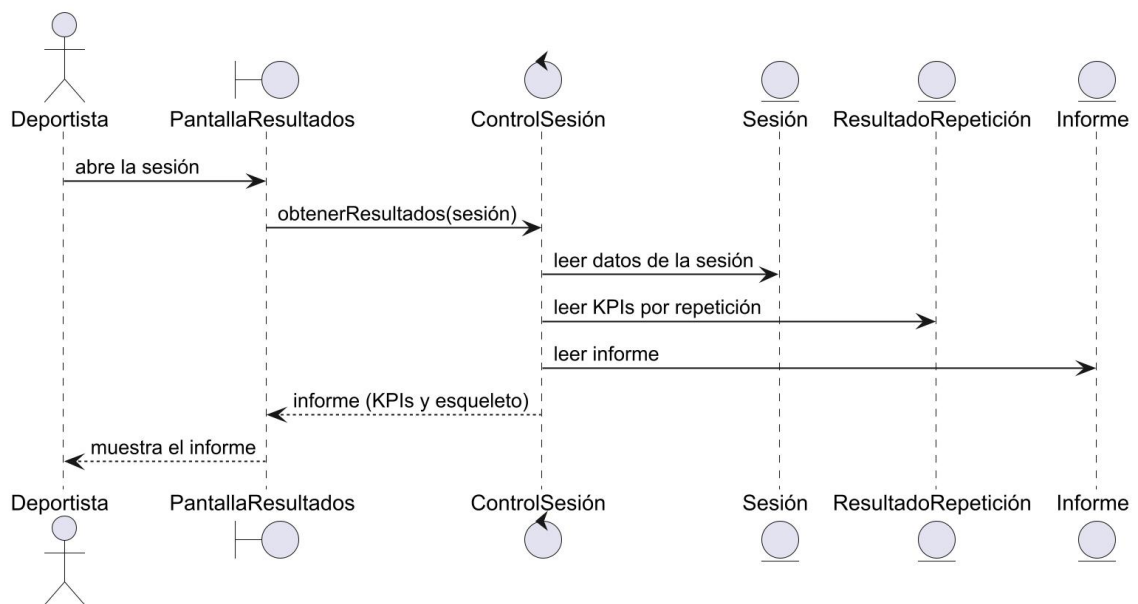


Figura 4. Diagrama de secuencia: consultar el informe de una sesión

Figura 5: Consultar el readiness (CU-10). Muestra cómo se obtiene el estado de forma: el servicio de readiness carga el historial reciente (sesiones, técnica y dolor), combina las cuatro dimensiones (fatiga, ACWR, dolor y técnica) y devuelve el valor con su desglose y el aviso principal.

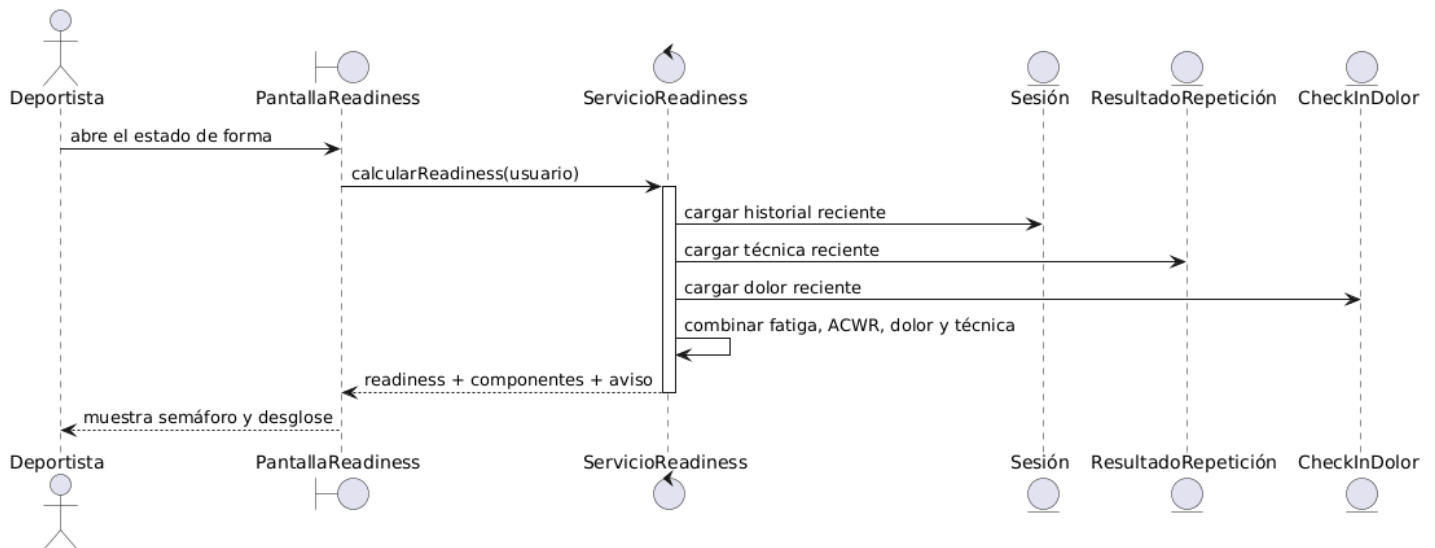


Figura 5. Diagrama de secuencia: consultar readiness

Figura 6: Consultar recomendaciones (CU-13). El control de recomendaciones reúne el estado actual del deportista (readiness, KPIs y dolores recientes), evalúa las condiciones que las activan y genera las recomendaciones aplicables, ordenadas por prioridad.

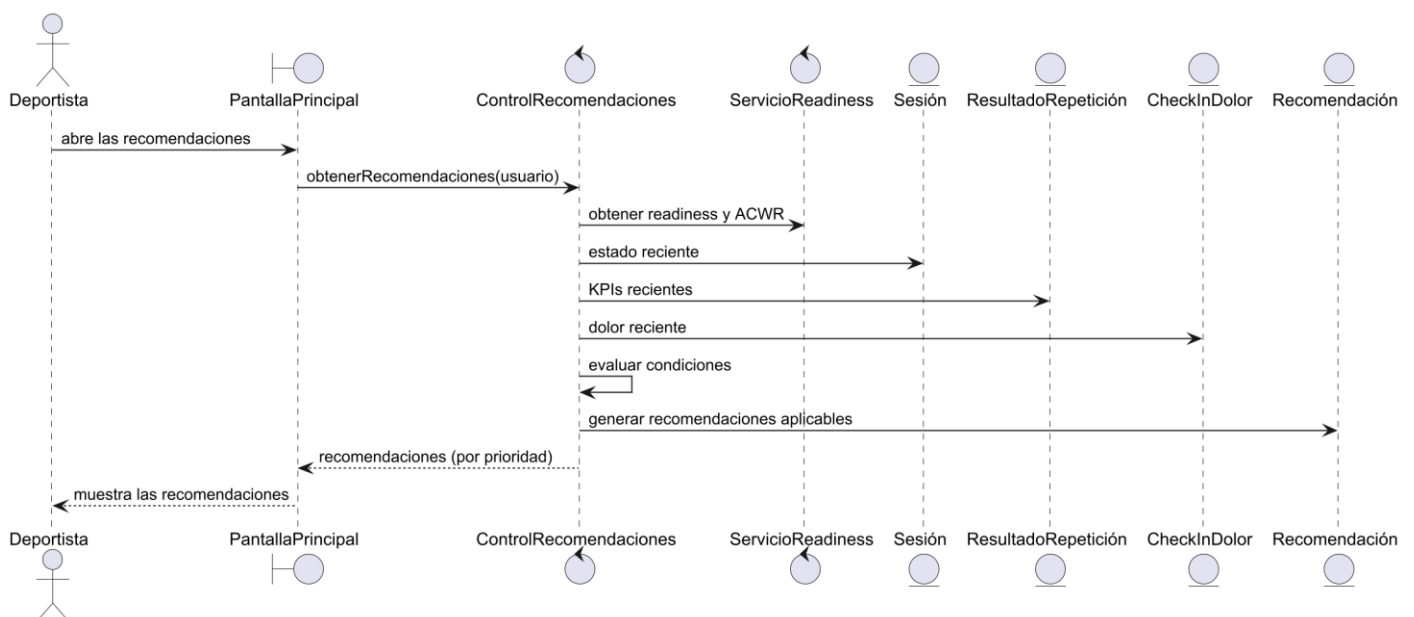


Figura 6. Diagrama de secuencia: consultar recomendaciones

Figura 7: Iniciar sesión (CU-03). El deportista introduce sus credenciales; el control de autenticación recupera el usuario, valida las credenciales y el estado de verificación de la cuenta y, según el resultado, concede el acceso o devuelve un error. Es el único diagrama con bifurcación (alt), que recoge los dos desenlaces.

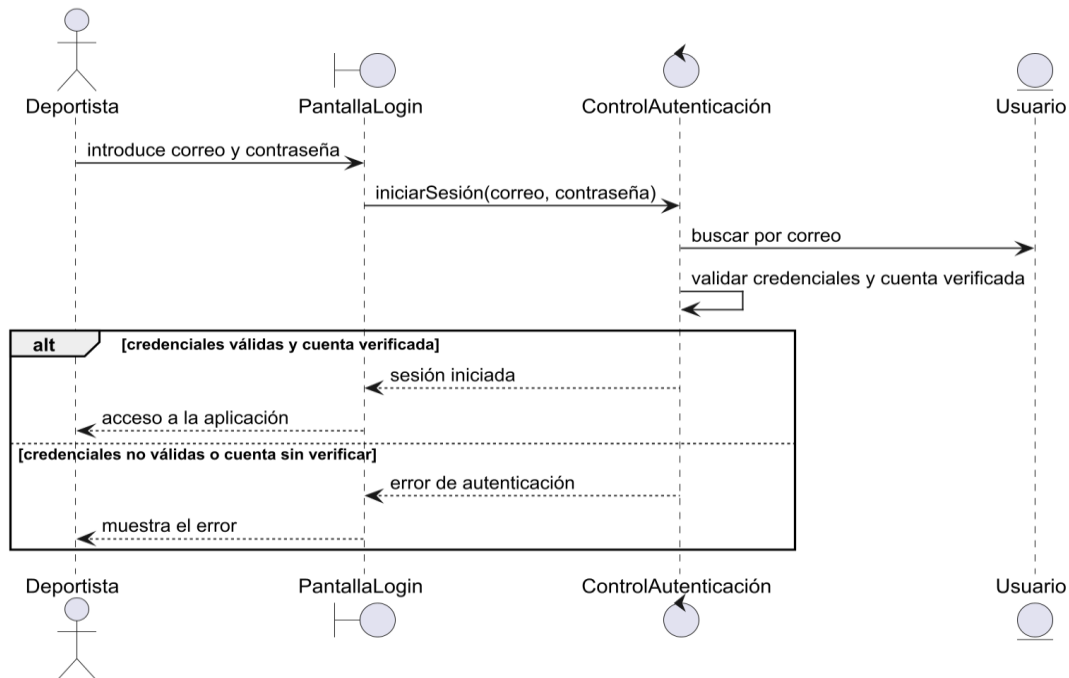


Figura 7. Diagrama de secuencia: iniciar sesión

Figura 8: Consultar bienestar (CU-12). Representa el módulo de bienestar: el control reúne las sesiones del usuario, obtiene el readiness asociado a cada una para construir la línea temporal y carga las incidencias registradas, devolviendo una visión conjunta de la evolución del estado de forma.

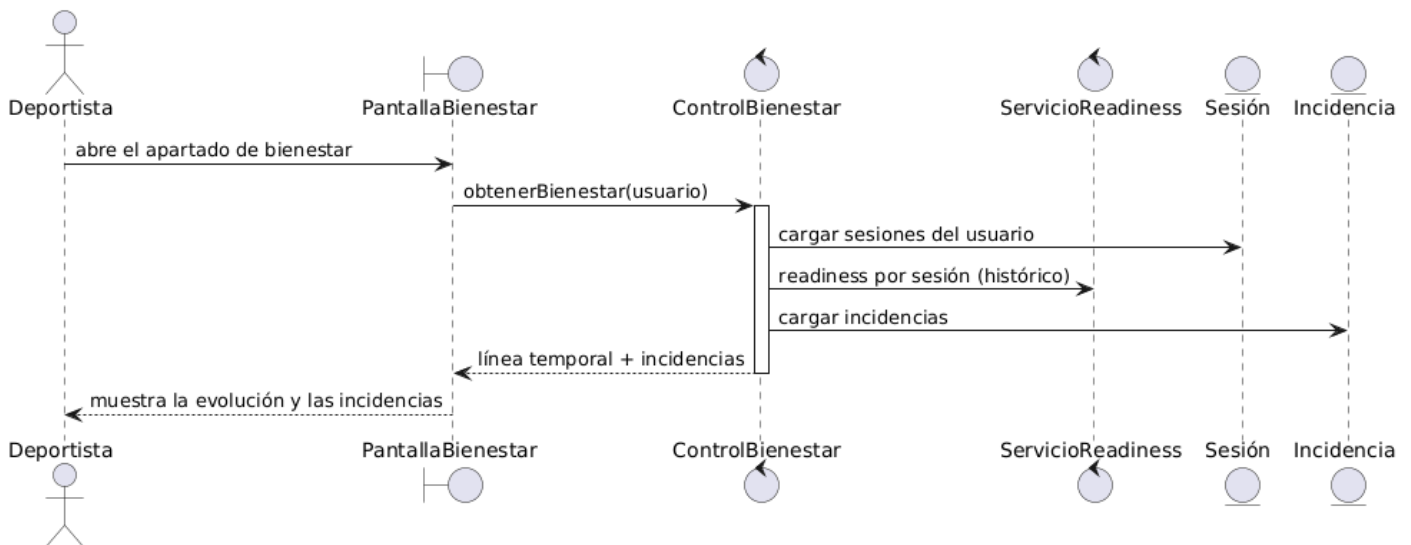


Figura 8. Diagrama de secuencia: consultar bienestar

Figura 9: Consultar análisis (CU-15). Muestra la analítica agregada: el control de analítica carga el historial de sesiones y sus KPIs y calcula los récords personales, la comparativa con la sesión anterior y el resumen semanal, que devuelve como un único conjunto de indicadores de progreso.

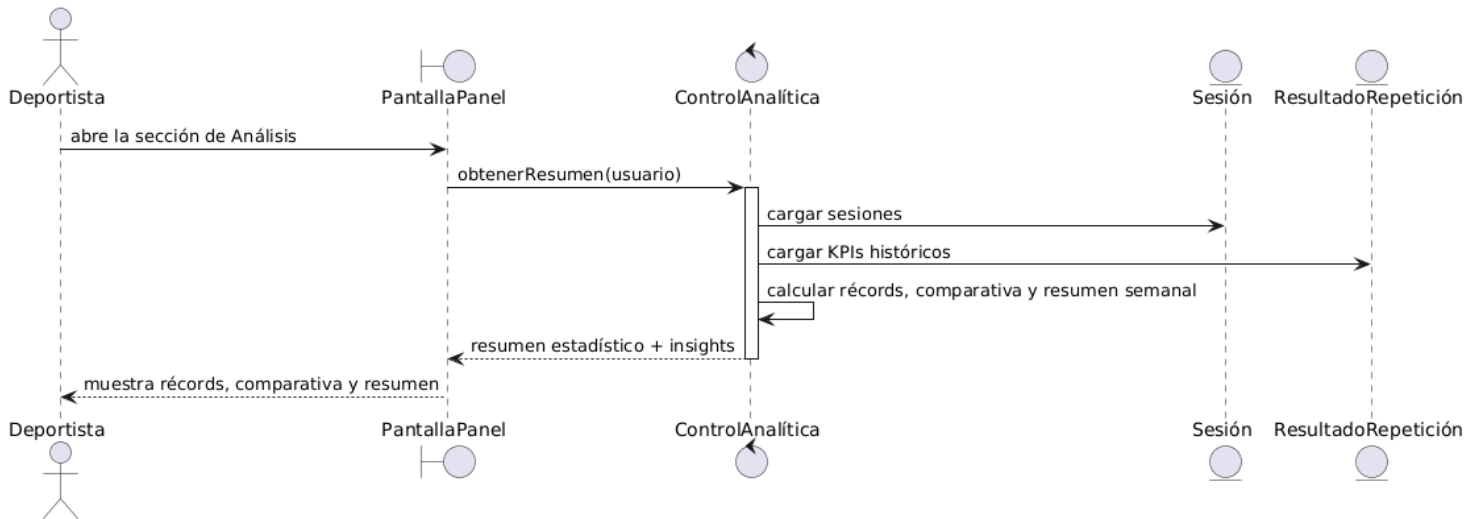


Figura 9. Diagrama de secuencia: consultar análisis

Figura 10: Exportar informe completo en PDF (CU-16). Distinto de los anteriores porque genera un documento: el control reúne todo el conjunto de datos del usuario, estos datos son: sesiones, KPIs y readiness, y luego produce el PDF con sus gráficos y recomendaciones, que se ofrece para descargar o compartir.

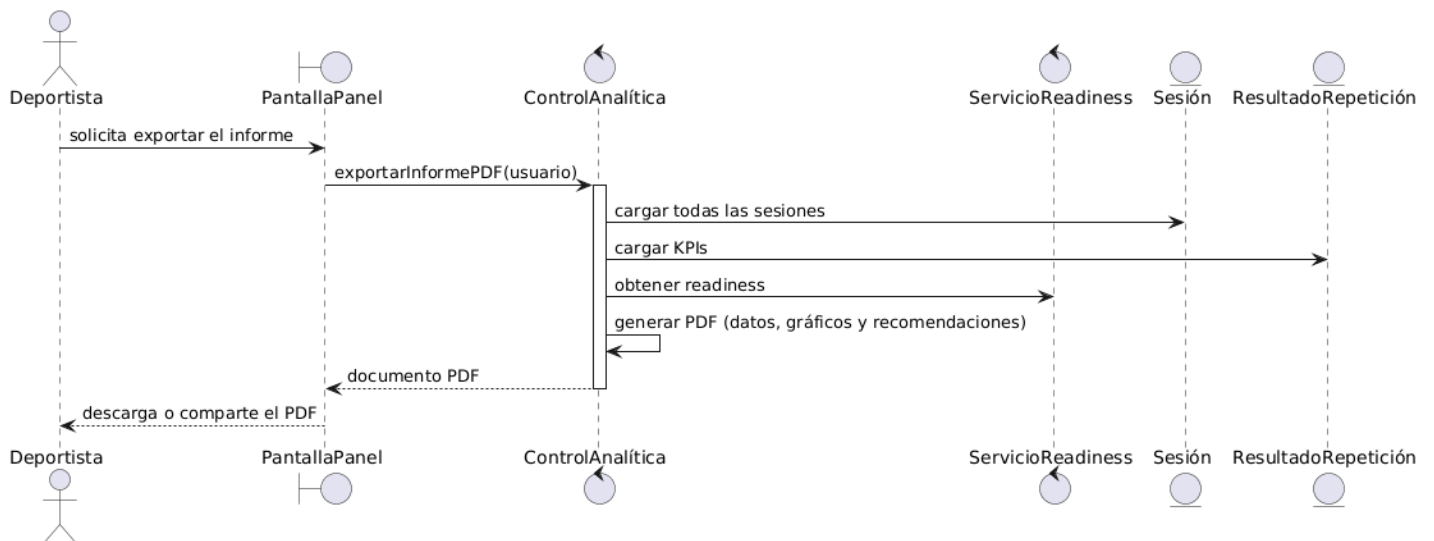


Figura 10. Diagrama de secuencia: exportar informe PDF

5 Diagrama de clases y paquetes de análisis

El diagrama de clases y paquetes ofrece la visión de más alto nivel del modelo de análisis: organiza en módulos cohesionados (paquetes) todas las clases identificadas durante la realización de los casos de uso y muestra las dependencias entre ellos. Cada paquete reúne las clases interfaz, control y entidad de un subsistema funcional, de modo que cada subsistema agrupa tanto la interacción con el usuario como la lógica y los conceptos del dominio sobre los que opera. Las dependencias entre paquetes se mantienen dirigidas y sin ciclos, lo que favorece la mantenibilidad y la evolución independiente de cada subsistema.

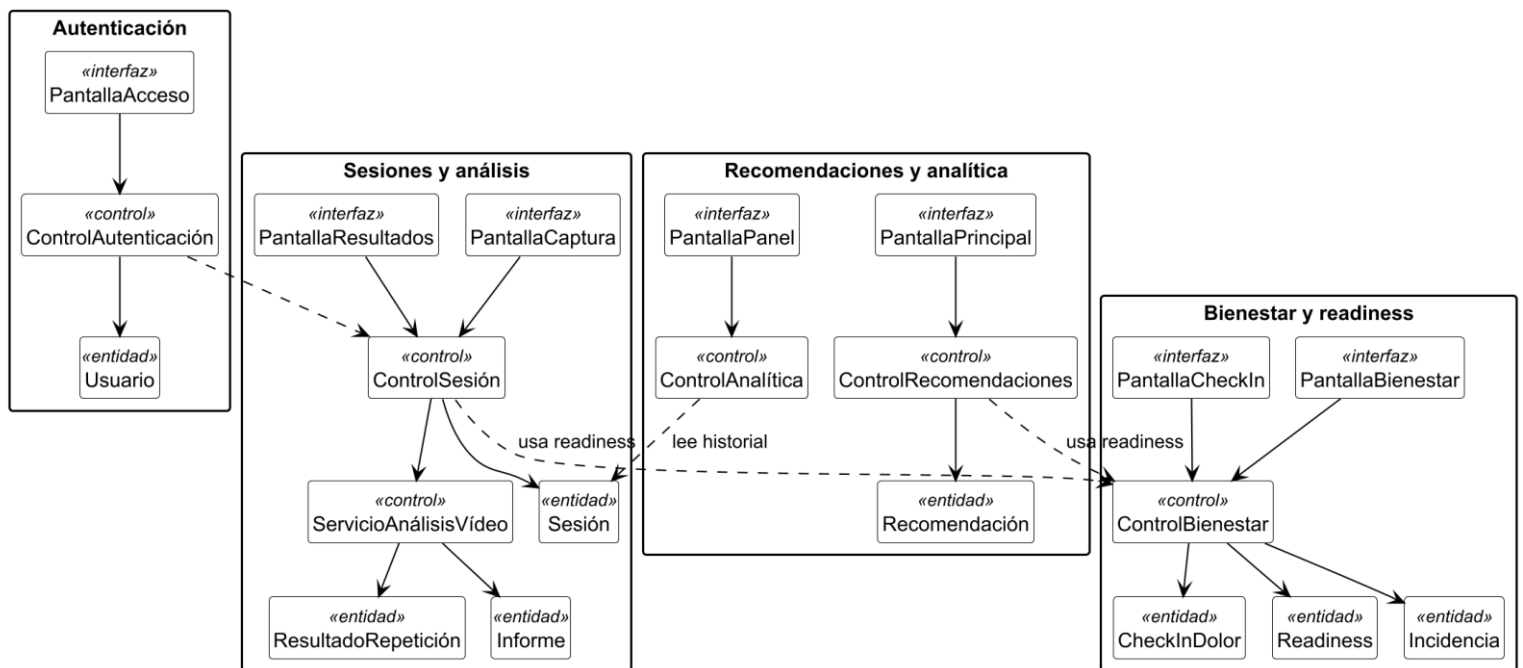


Figura 11. Diagrama de paquetes de análisis

Las dependencias entre subsistemas son escasas y están bien dirigidas. Sesiones y análisis y Recomendaciones y analítica dependen de Bienestar y readiness, porque reutilizan el cálculo del readiness; Recomendaciones y analítica consulta además el historial de Sesiones y análisis. Autenticación es independiente del resto. No existe ninguna dependencia cíclica, lo que indica un acoplamiento bajo y una buena cohesión de cada subsistema.

Con ello finaliza el modelo de análisis, que ha descrito la estructura conceptual del sistema y la colaboración entre sus clases con independencia de la tecnología. El resto del anexo aborda ya el diseño: las decisiones y la estructura concretas con las que se ha construido MoveInsight.

6 Arquitectura del sistema

Con este capítulo se inicia la fase de diseño y se describe la arquitectura de MoveInsight, donde se verá la organización de alto nivel del sistema y las decisiones estructurales que la componen. Se presenta en tres niveles complementarios: primero el estilo arquitectónico adoptado y su justificación; después el modelo C4, que ofrece vistas a distintos grados de detalle (contexto, contenedores y componentes); y, por último, las principales decisiones arquitectónicas, formuladas como registros de decisión y vinculadas a los atributos de calidad que las motivan.

6.1 Estilo arquitectónico

MoveInsight adopta una arquitectura cliente-servidor distribuida, en la que una aplicación móvil consume, a través de una API REST sobre HTTP, los servicios de un backend que concentra el análisis biomecánico y la persistencia. Ambos extremos se organizan internamente por capas, lo que separa la presentación, la lógica de aplicación y el acceso a los datos, y mantiene las dependencias dirigidas hacia el interior del sistema.

El cliente Android sigue el patrón MVVM combinado con Clean Architecture [Martin, 2017], estructurado en tres capas. La capa de presentación se construye con Jetpack Compose y un ViewModel por pantalla, que gestiona el estado y reacciona a los eventos de la interfaz. La capa de dominio define las abstracciones de acceso a datos (los repositorios) de forma independiente de su implementación. La capa de datos las materializa mediante Retrofit/OkHttp para el consumo de la API y DataStore para la persistencia local del token de sesión. La inyección de dependencias con Hilt desacopla las capas y facilita su sustitución y prueba.

El servidor se organiza también por capas con FastAPI sobre el servidor ASGI Uvicorn: los routers exponen los puntos de entrada de la API (capa de presentación), los services concentran la lógica de negocio, donde podemos encontrar el cálculo del readiness, el motor de recomendaciones y, sobre todo, la canalización de análisis del vídeo. La capa de datos accede a la base de datos relacional MariaDB mediante el ORM SQLAlchemy. El análisis del vídeo, que combina MediaPipe (estimación de pose), un modelo BiLSTM (segmentación de fases) y modelos XGBoost (clasificación de los KPIs por repetición), es un análisis pesado, por lo que se ejecuta de forma asíncrona en segundo plano: la subida responde de inmediato y el cliente sondea el estado de la sesión hasta su finalización.

Por último, el sistema se despliega en contenedores mediante Docker, con Nginx como proxy inverso a la entrada, lo que proporciona un despliegue reproducible y un punto único para la terminación de las conexiones seguras.

La elección de una arquitectura distribuida cliente-servidor, frente a una aplicación monolítica que ejecutara todo el análisis en el propio dispositivo, responde a varios atributos de calidad. El procesamiento de visión por ordenador y aprendizaje automático es exigente y depende de modelos voluminosos que se cargan una sola vez en memoria; centralizarlo en el servidor garantiza resultados consistentes con independencia de la potencia del teléfono, permite actualizar los modelos sin publicar una nueva versión de la aplicación y mantiene el cliente ligero, lo que favorece el rendimiento y la fiabilidad del análisis. Además, situar la lógica sensible y la validación en el servidor refuerza la seguridad. La organización por capas y el uso de Clean Architecture en el cliente persiguen la mantenibilidad y la capacidad de evolución, separando responsabilidades y aislando la interfaz de las fuentes de datos. El procesamiento asíncrono, por su parte, preserva la respuesta fluida de la aplicación pese al coste del análisis.

Se descartó una arquitectura de microservicios por resultar desproporcionada para un proyecto de este alcance, desarrollado por una sola persona. Las decisiones puntuales que concretan este estilo como la elección de la base de datos, el mecanismo de autenticación, el procesamiento asíncrono o el despliegue se detallan, con su motivación, en el apartado 6.3 Decisiones .

6.2 Modelo C4

El modelo C4 describe la arquitectura mediante vistas a niveles de detalle crecientes. Se presentan los siguientes diagramas: el diagrama de contexto, que sitúa el sistema frente a sus usuarios y sistemas externos; el de contenedores, que muestra las unidades desplegadas y su comunicación; y el de componentes, que desgrana la estructura interna del contenedor principal, la API.

Nivel 1 (figura 12): Contexto. MoveInsight es utilizado por el deportista a través de la aplicación móvil, y su único sistema externo es el servidor de correo, al que envía los mensajes de verificación de cuenta y de restablecimiento de contraseña

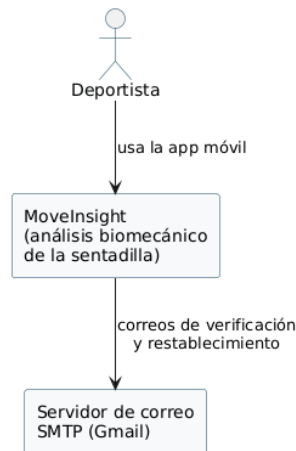


Figura 12. Diagrama C4: Nivel de contexto

Nivel 2 (figura 13): Contenedores. El sistema se despliega en un único servidor mediante Docker Compose. La aplicación móvil se comunica por HTTPS con Nginx, único contenedor expuesto a internet, que actúa como proxy inverso (con certificado TLS de Let's Encrypt) hacia la API. La API concentra la lógica de negocio y la canalización de análisis, persiste en MariaDB y guarda los vídeos e informes en volúmenes del servidor.

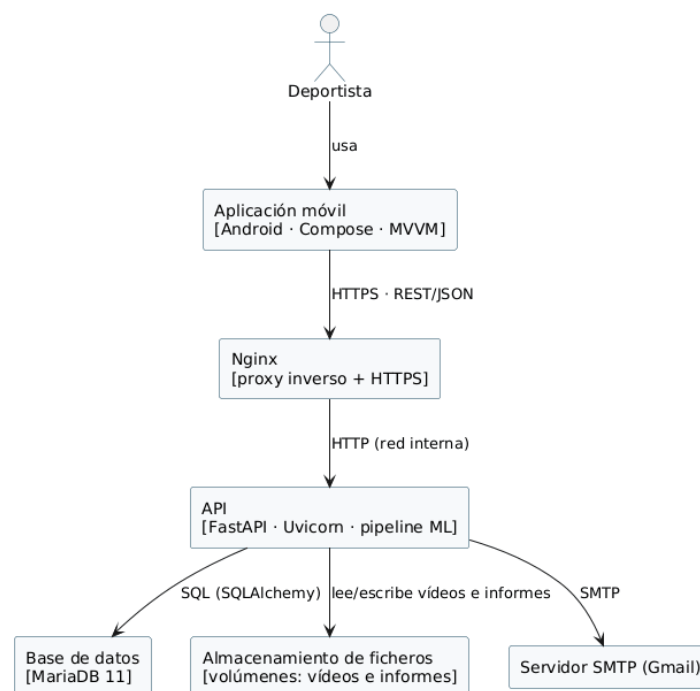


Figura 13. Diagrama C4: Nivel de contenedores

Nivel 3 (figura 14): Componentes. Dentro de la API, los routers exponen los puntos de entrada y delegan en los servicios de negocio: el servicio de readiness, el motor de recomendaciones y el servicio de análisis de vídeo, que encadena la estimación de pose (MediaPipe), la segmentación de fases (BiLSTM) y la clasificación de los KPIs (XGBoost). Los componentes transversales gestionan la autenticación y el envío de correo, y el acceso a datos media entre los servicios y MariaDB.

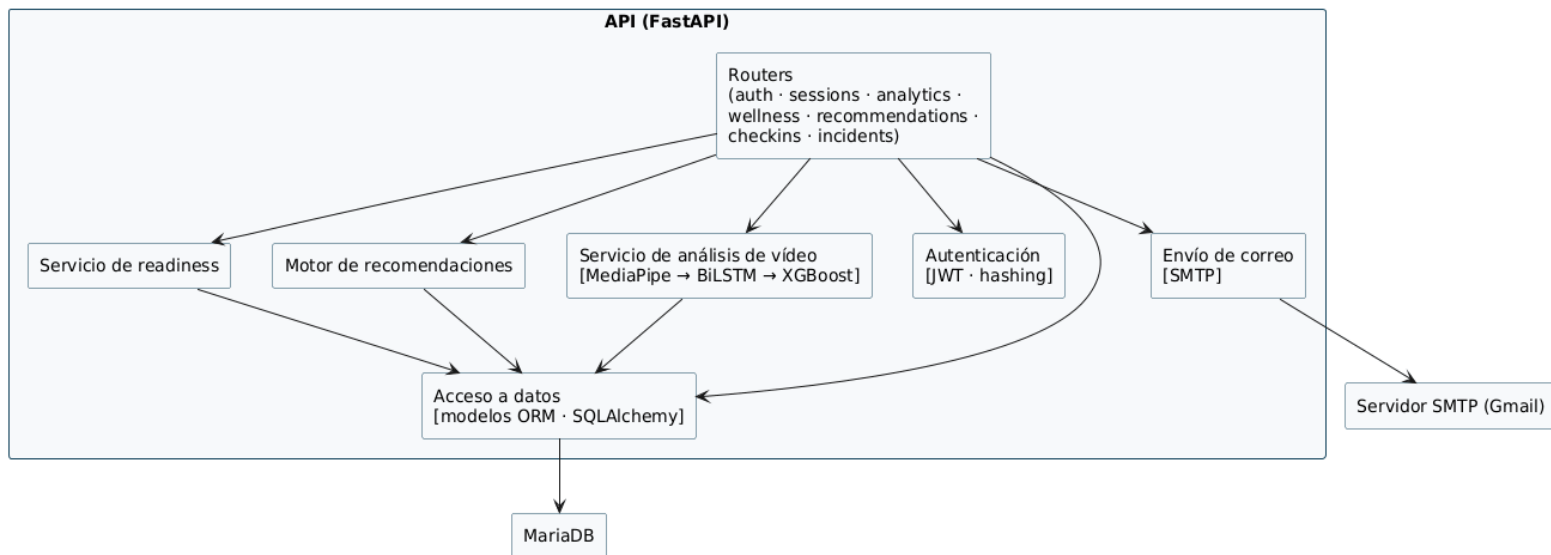


Figura 14. Diagrama C4: Nivel de componentes

6.3 Decisiones arquitectónicas

Este apartado recoge las decisiones arquitectónicas más relevantes en forma de registros de decisión (ADR). Cada registro documenta el contexto que plantea la decisión, la opción adoptada, las alternativas descartadas y sus consecuencias, y se vincula con los requisitos no funcionales del Anexo I que la motivan. Al final, una matriz resume esa relación.

ADR-01	Análisis biomecánico en el servidor
Contexto	El análisis exige visión por computador y modelos de aprendizaje automático voluminosos, mientras que los dispositivos móviles ofrecen una capacidad de cómputo dispar.
Decisión	Centralizar todo el análisis en un <i>backend</i> ; la aplicación móvil actúa como cliente que captura el vídeo y consulta los resultados (arquitectura cliente-servidor con API REST).
Alternativas descartadas	Ejecutar el análisis en el propio dispositivo (<i>on-device</i>).
Atributos de calidad	NFR-03 Precisión (resultados consistentes entre dispositivos), NFR-01 Rendimiento (cliente ligero), NFR-08 Mantenibilidad (actualizar modelos sin republicar la app), NFR-05 Seguridad (lógica y validación en el servidor).
Consecuencias	Requiere conectividad e introduce latencia de red; concentra la carga en el servidor, lo que motiva la decisión ADR-02.

Tabla 5. ADR-01

ADR-02	Procesamiento asíncrono del vídeo
Contexto	El análisis de un vídeo tarda varios segundos; resolverlo dentro de la petición bloquearía al usuario y arriesgaría tiempos de espera agotados.
Decisión	La subida crea la sesión en estado “encolada” y lanza el análisis en segundo plano (FastAPI <i>BackgroundTasks</i>), respondiendo de inmediato; el cliente sondea el estado hasta “completada” o “fallida”.
Alternativas descartadas	Procesamiento síncrono en la propia petición; cola de tareas externa (Celery/Redis).
Atributos de calidad	NFR-01 Rendimiento, NFR-04 Usabilidad (la app no se bloquea), NFR-02 Fiabilidad (un fallo se refleja como “fallida” sin afectar al resto).
Consecuencias	El cliente debe sondear el estado; se renunció a una cola externa por no justificar su complejidad operativa para el alcance del proyecto.

Tabla 6. ADR-02

ADR-03	Persistencia en base de datos relacional
Contexto	Los datos del dominio están fuertemente relacionados (usuario–sesión–resultado–check-in–incidencia) y requieren integridad referencial.
Decisión	Emplear la base de datos relacional MariaDB, accedida mediante el ORM SQLAlchemy.
Alternativas descartadas	Base de datos documental NoSQL.
Atributos de calidad	NFR-02 Fiabilidad (integridad referencial), NFR-08 Mantenibilidad (esquema explícito), NFR-06 Privacidad (control estructurado de los datos personales).
Consecuencias	El esquema es rígido y los cambios exigen migraciones, lo que resulta asumible dado lo estructurado del dominio.

Tabla 7. ADR-03

ADR-04	Autenticación sin estado mediante JWT
Contexto	Una API REST consumida por un cliente móvil se beneficia de no mantener estado de sesión en el servidor.
Decisión	Autenticación con tokens JWT firmados (acceso y refresco), contraseñas almacenadas con <i>hash</i> y verificación de correo mediante código.
Alternativas descartadas	Sesiones con estado basadas en cookies de servidor.
Atributos de calidad	NFR-05 Seguridad, NFR-01 Rendimiento (servicio sin estado), NFR-06 Privacidad.
Consecuencias	La revocación inmediata de un token es más compleja; se soluciona con su caducidad y el mecanismo de refresco.

Tabla 8. ADR-04

ADR-05	Cliente Android nativo (Compose + MVVM + Clean Architecture)
Contexto	La aplicación necesita acceso fluido a la cámara y la galería y una interfaz reactiva, y la desarrolla una sola persona.
Decisión	Aplicación nativa Android con Jetpack Compose, patrón MVVM y Clean Architecture, inyección de dependencias con Hilt y consumo de la API con Retrofit.
Alternativas descartadas	Un framework multiplataforma (Flutter, React Native).
Atributos de calidad	NFR-04 Usabilidad, NFR-08 Mantenibilidad y testabilidad (separación en capas), NFR-07 Compatibilidad (Android).
Consecuencias	El sistema queda limitado a Android; a cambio, se obtiene integración nativa y un ecosistema de herramientas maduro.

Tabla 9. ADR-05

ADR-06	Despliegue contenedorizado con Nginx y HTTPS
Contexto	El sistema se despliega en un único servidor de la Universidad y necesita reproducibilidad y acceso seguro.
Decisión	Orquestación con Docker Compose (base de datos, API, Nginx y certbot); Nginx como proxy inverso con TLS de Let's Encrypt, siendo el único servicio expuesto a internet.
Alternativas descartadas	Despliegue directo sobre el <i>host</i> ; plataforma como servicio (PaaS) en la nube.
Atributos de calidad	NFR-08 Mantenibilidad y portabilidad, NFR-05 Seguridad (HTTPS y red interna aislada), NFR-02 Fiabilidad (políticas de reinicio y healthchecks).
Consecuencias	El escalado queda limitado al servidor único (escalado vertical), suficiente para el alcance del proyecto.

Tabla 10. ADR-06

6.3.1 Matriz de decisiones × atributos de calidad

La siguiente matriz cruza cada decisión arquitectónica con los atributos de calidad sobre los que influye, lo que permite comprobar de un vistazo qué decisiones respaldan cada objetivo de calidad.

	NFR-01 Rend.	NFR-02 Fiab.	NFR-03 Prec.	NFR-04 Usab.	NFR-05 Seg.	NFR-06 Priv.	NFR-07 Comp.	NFR-08 Mant.
ADR-01	X		X		X			X
ADR-02	X	X		X				
ADR-03		X				X		X
ADR-04	X				X	X		
ADR-05				X			X	X
ADR-06		X			X			X

Tabla 11. Matriz de decisiones x NFR

7 Diseño de la funcionalidad

Tras definir la arquitectura, este capítulo desarrolla el diseño de la funcionalidad: las clases concretas que la implementan, su interacción detallada, su comportamiento dinámico y los patrones de diseño aplicados. A diferencia del modelo de análisis, que se mantenía independiente de la tecnología, el diseño incorpora ya las decisiones del capítulo anterior.

7.1 Clases de diseño

Las clases de diseño refinan las clases de análisis incorporando el detalle propio de la implementación: los tipos de los atributos, las operaciones y las clases que introduce la arquitectura por capas.

El diagrama recoge, sobre una misma vista, las dos mitades del sistema y sus respectivas capas: en el cliente Android, la presentación (pantallas y ViewModels), el dominio (casos de uso y repositorios) y los datos (acceso a la API y almacenamiento local); y en el servidor FastAPI, la capa de API (routers), los servicios de negocio, la canalización de análisis del vídeo y la persistencia. Así se muestra el recorrido completo de una petición, desde la interfaz hasta la base de datos.

Antes del diagrama, la siguiente tabla traza la correspondencia entre cada clase de análisis y la clase o componente que la realiza en el diseño, de modo que pueda seguirse la continuidad entre ambos modelos.

Clase de análisis	Tipo	Implementación
PantallaLogin	interfaz	LoginScreen
PantallaRegistro	interfaz	RegisterScreen
PantallaVerificaciónCorreo	interfaz	VerifyEmailScreen
PantallaRecuperarContraseña	interfaz	ForgotPasswordScreen / ResetPasswordScreen
PantallaCambiarContraseña	interfaz	ChangePasswordScreen
PantallaOnboarding	interfaz	OnboardingScreen
PantallaPrincipal	interfaz	HomeScreen
PantallaPanel	interfaz	DashboardScreen
PantallaCaptura	interfaz	CaptureScreen
PantallaResultados	interfaz	ResultsScreen
PantallaVídeoEsqueleto	interfaz	SkeletonVideoScreen
PantallaReadiness	interfaz	ReadinessBreakdownScreen

PantallaBienestar	interfaz	WellnessScreen
PantallaFactoresIncidencia	interfaz	IncidentFactorsScreen
PantallaCheckInDolor	interfaz	PainCheckInScreen
ControlAutenticación	control	routers/auth.py, auth.py, email_utils.py
ControlSesión	control	routers/sessions.py
ServicioAnálisisVídeo	control	services/video.py
ServicioReadiness	control	services/readiness.py
ControlBienestar	control	routers/wellness.py, incidents.py, checkins.py
ControlRecomendaciones	control	routers/recommendations.py, services/recommendations.py
ControlAnalítica	control	routers/analytics.py

Tabla 12. Clases de análisis y su implementación en el diseño

En la figura 15 se observa el diagrama de clases de diseño:

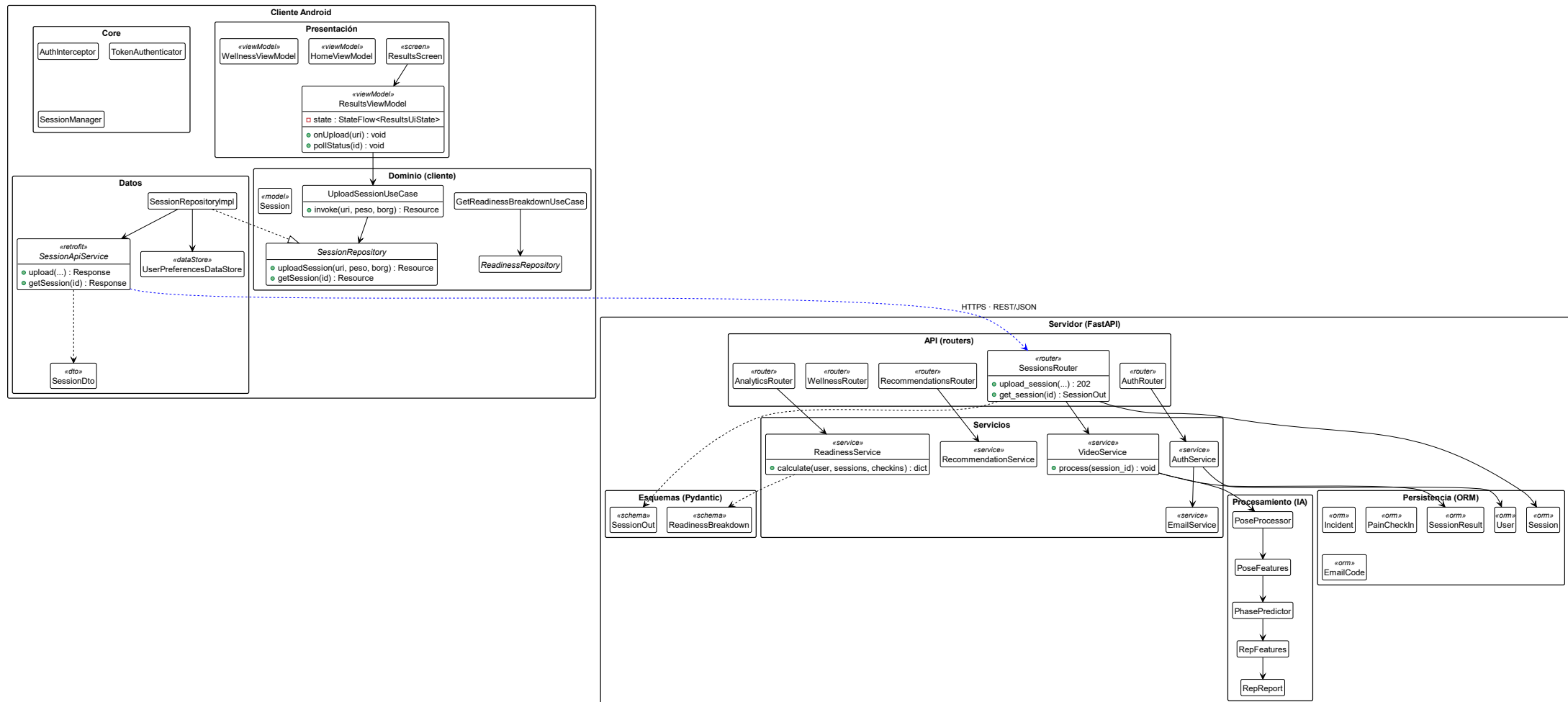


Figura 15. Diagrama de clases de diseño

7.1.1 Clases de infraestructura

Junto a las clases de dominio, la arquitectura por capas introduce un conjunto de clases de infraestructura que no representan conceptos del problema, sino que dan soporte técnico a la solución: gestionan la presentación, la comunicación con la API, el acceso a los datos y la coordinación entre capas.

El siguiente diagrama las recoge, y la tabla posterior resume la responsabilidad de cada una

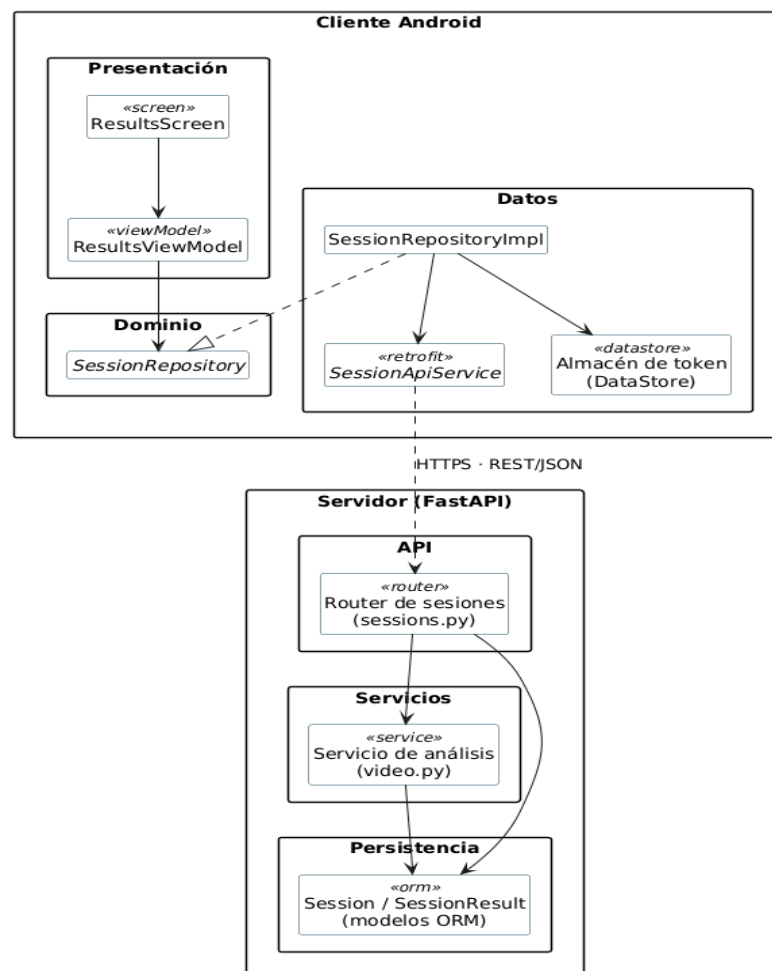


Figura 16. Diagrama de clases de infraestructura

En la siguiente tabla se puede observar las responsabilidades de cada capa junto con sus capas principales:

Capa	Clases principales	Responsabilidad
Cliente · Presentación	· Screens (ResultsScreen...) + ViewModels (ResultsViewModel...)	Renderizan la interfaz con Compose y gestionan el estado de cada pantalla (un ViewModel por pantalla).
Cliente · Dominio	· UseCases, interfaces Repository, modelos	Encapsulan la lógica de aplicación con independencia de la infraestructura.
Cliente · Datos	· SessionRepositoryImpl, SessionApiService (Retrofit), UserPreferencesDataStore	Implementan los repositorios consumiendo la API y persisten localmente el token de sesión.
Cliente · Core	AuthInterceptor, TokenAuthenticator, SessionManager	Servicios transversales: añaden el token a las peticiones y gestionan la expiración de la sesión.
Servidor · API	Routers (auth, sessions, analytics, wellness, checkins, incidents, recommendations)	Exponen los endpoints, validan la petición y delegan en los servicios.
Servidor · Servicios	· VideoService, ReadinessService, RecommendationService, AuthService, EmailService	Concentran la lógica de negocio.
Servidor · Procesamiento	· PoseProcessor, PhasePredictor, RepFeatures, RepReport, Visualization	Canalización de análisis del vídeo (pose, fases, KPIs e informe).
Servidor · Persistencia	· Modelos ORM (User, Session, SessionResult, Incident, PainCheckIn, EmailCode)	Mapean las entidades a las tablas de MariaDB mediante SQLAlchemy.
Servidor · Esquemas	· DTOs Pydantic (SessionOut, ReadinessBreakdown...)	Definen y validan el contrato de entrada/salida de la API.

Tabla 13. Clases del diseño por capas

7.2 Diagramas de secuencia de diseño

Los diagramas de secuencia de diseño detallan la interacción entre las clases reales del sistema, organizadas por capas, e incorporan el paso por la red entre cliente y servidor. A diferencia de los del apartado 4 que trabajaban con clases de análisis abstractas, estos reflejan ya las clases concretas y los *endpoints* reales de la API. Se presentan los tres flujos más representativos del diseño: el inicio de sesión, la subida y análisis asíncrono del vídeo, y la consulta del informe con sondeo.

Diagrama de diseño (figura 17): Iniciar sesión (CU-03). Recorre las capas del cliente hasta la API: el ViewModel delega en el repositorio, que llama al ApiService; el router valida las credenciales contra el modelo User y devuelve los tokens, que el repositorio persiste en DataStore.

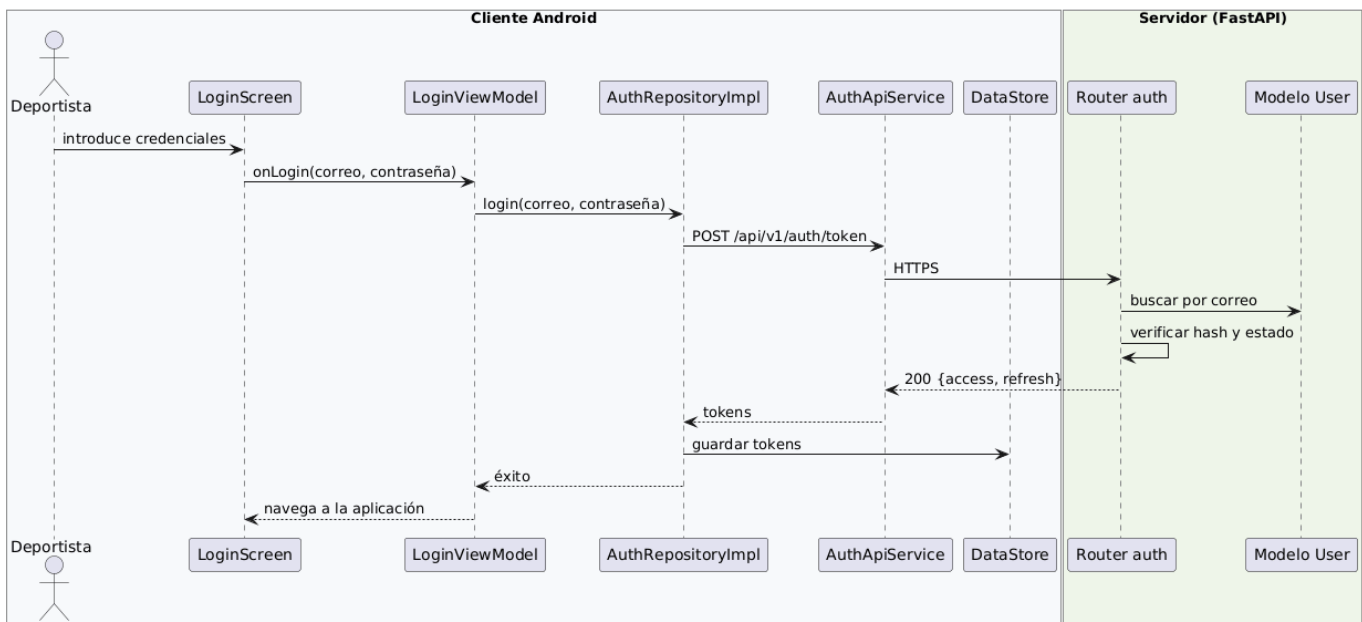


Figura 17. Diagrama de diseño: iniciar sesión

Diagrama de diseño (figura 18): Subir vídeo y análisis asíncrono (CU-07 + CU-11). El núcleo del sistema. El router crea la sesión, lanza el análisis como tarea en segundo plano y responde con 202 de inmediato. En segundo plano, el servicio de análisis ejecuta la canalización de ML y actualiza la sesión.

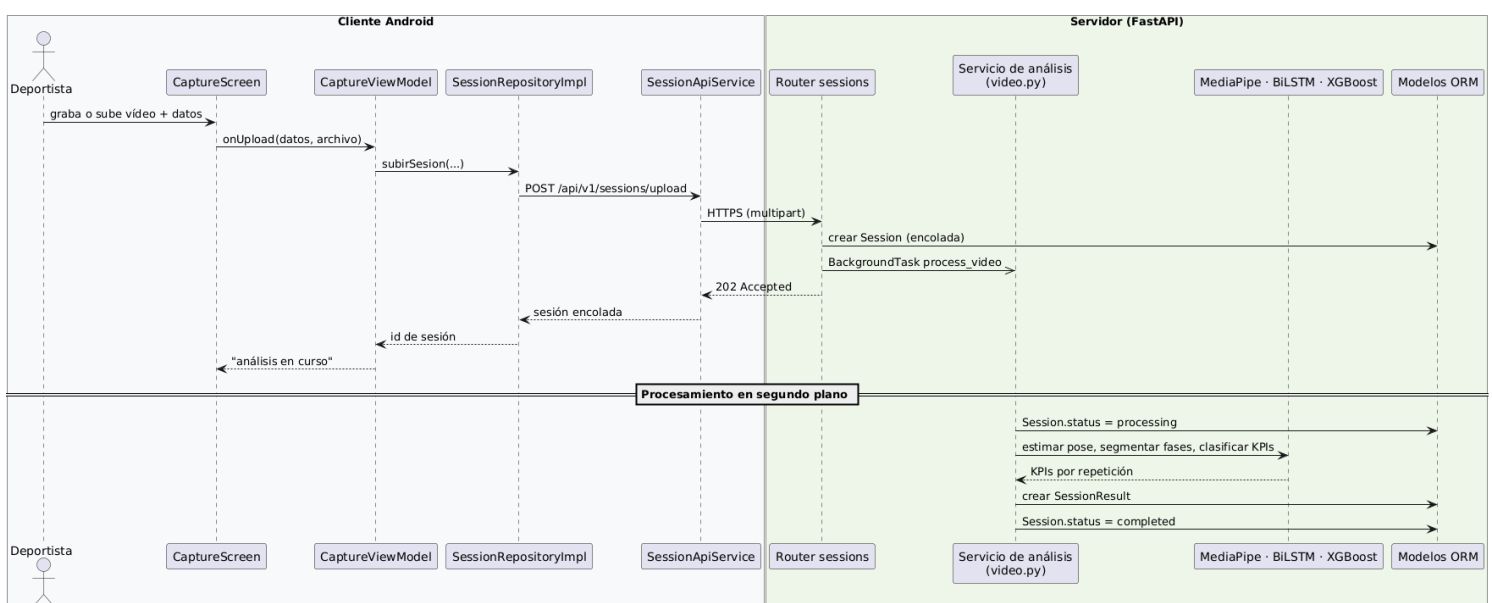


Figura 18. Diagrama de diseño: subir vídeo y análisis asíncrono

Diagrama de diseño (figura 19): Consultar el informe con sondeo (CU-08). El ViewModel sondea periódicamente el mismo endpoint GET /sessions/{id}, que devuelve el detalle de la sesión (estado y, cuando ha terminado, los KPIs). Al alcanzar el estado “completada”, muestra el informe.

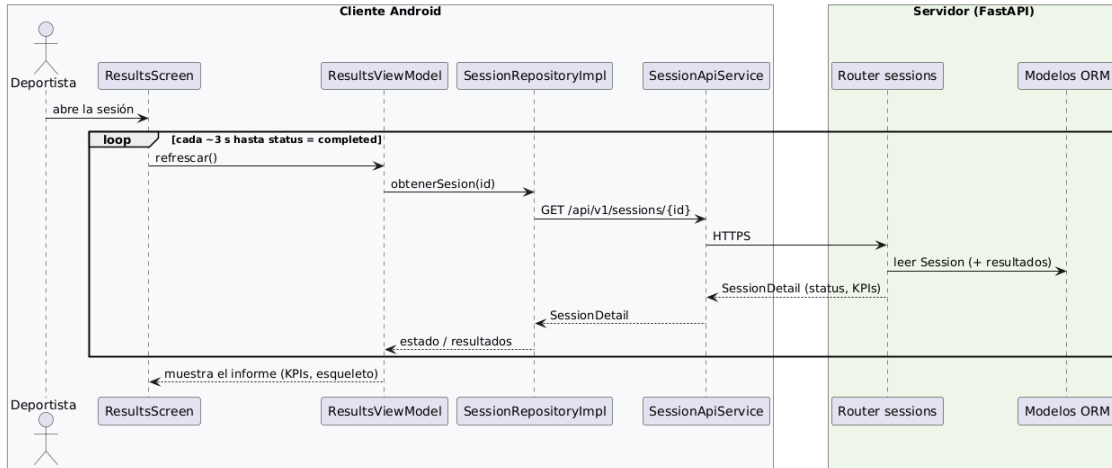


Figura 19. Diagrama de diseño: consultar el informe con sondeo

7.3 Diagramas de estados y de actividad

Los diagramas de estados y de actividad se incluyen únicamente donde aportan información que las vistas anteriores no recogen: el ciclo de vida de un objeto con estados bien definidos o un algoritmo con bifurcaciones relevantes. En MoveInsight se han identificado tres casos que se muestran a continuación.

Diagrama de estados (figura 20): Ciclo de vida de una sesión. El atributo status de la sesión rige todo el procesamiento asíncrono. Una sesión nace “encolada” al subir el vídeo, pasa a “procesando” cuando el servicio toma la tarea y termina en “completada” o “fallida”. Ambos estados finales son terminales.

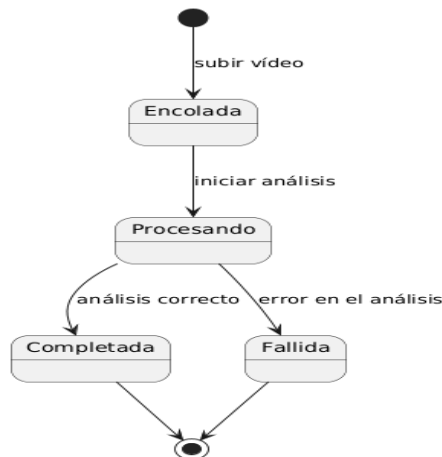


Figura 20. Diagrama de estados: ciclo de vida de una sesión

Diagrama de actividad (figura 21): Canalización de análisis del vídeo. Detalla el algoritmo que ejecuta el servicio de análisis en segundo plano (lo que en el diagrama de secuencia de subir vídeo (Figura 3) era un solo mensaje). La bifurcación clave es el perfil de grabación: en vista lateral se excluyen el valgo y la simetría, no medibles de forma fiable.

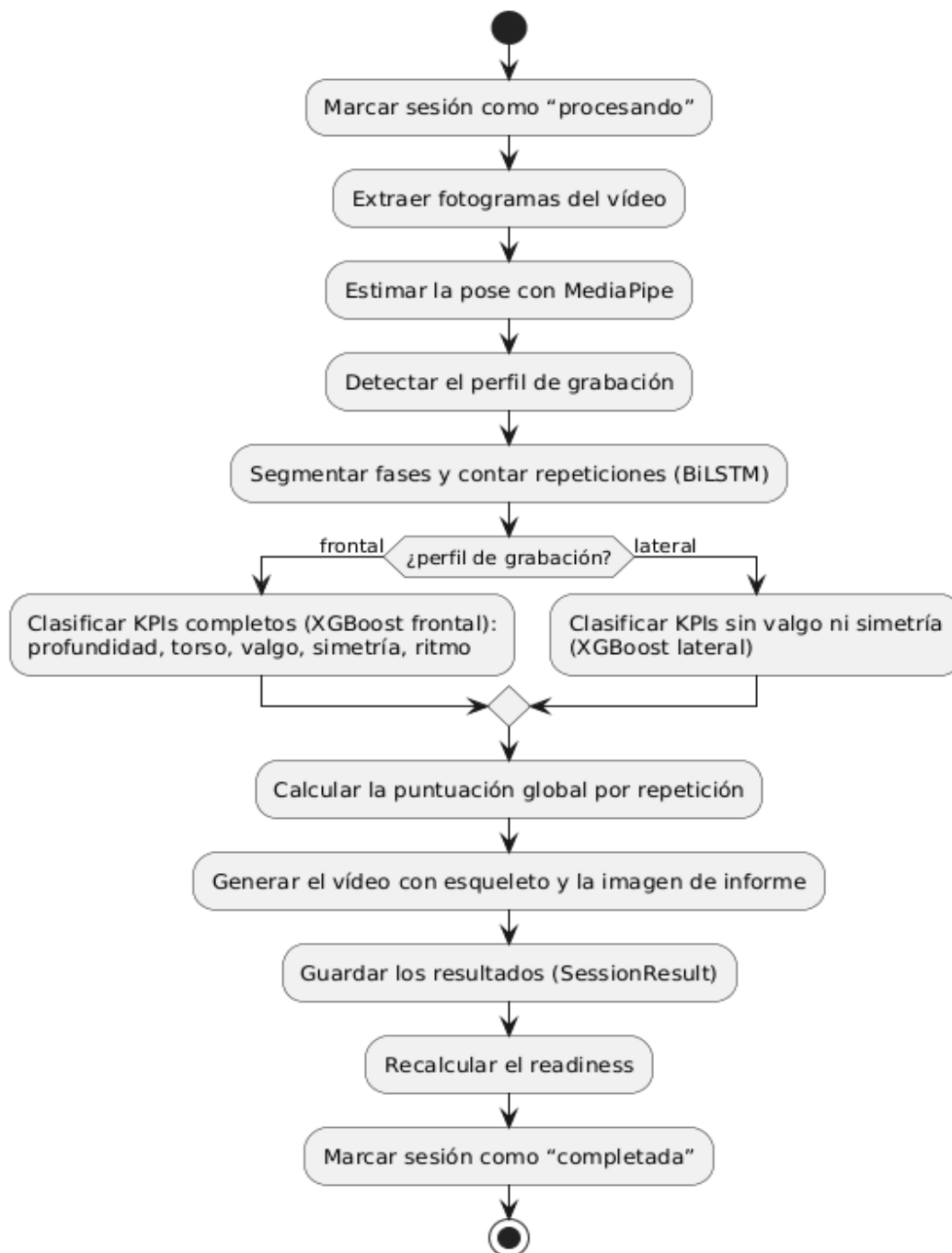


Figura 21. Diagrama de actividad: canalización de análisis del vídeo

Cualquier error durante el proceso marca la sesión como “fallida” (estado terminal del diagrama anterior), sin afectar al resto de sesiones.

Diagrama de actividad (figura 22): Cálculo del readiness. Muestra la lógica de ponderación adaptativa: el peso de cada señal (fatiga, carga, dolor, técnica) depende de la madurez del histórico del usuario, lo que evita dar un valor poco fiable cuando aún hay pocos datos.

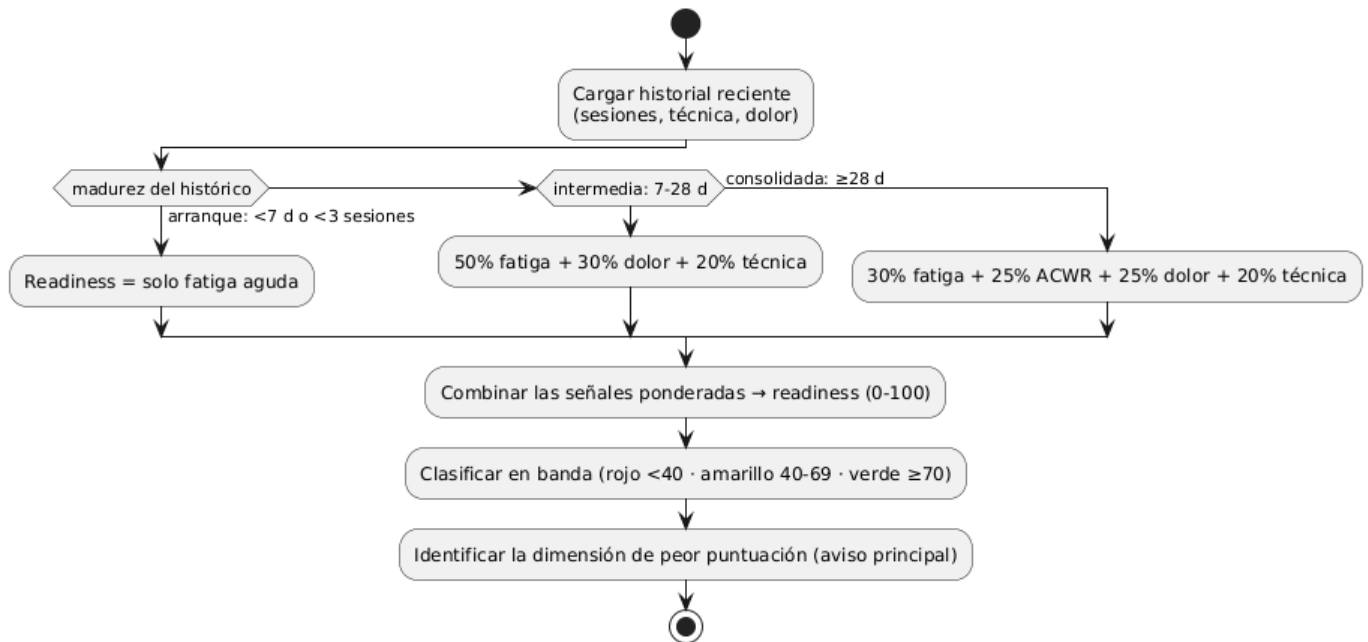


Figura 22. Diagrama de actividad: cálculo del readiness

El resto de las operaciones del sistema son lineales (sin estados ni ramas relevantes), por lo que no se les añade diagrama, en coherencia con el criterio de incluirlos solo donde aportan.

7.4 Patrones de diseño

El diseño de MoveInsight se apoya en patrones consolidados, tanto patrones GoF y arquitectónicos como principios GRASP de asignación de responsabilidades. Este apartado recoge únicamente los aplicados de forma efectiva en el sistema, indicando dónde aparecen y qué problema resuelven; no se incluyen patrones que no estén realmente presentes en la implementación.

Patrón	Tipo	Dónde se aplica	Problema que resuelve
MVVM	Arquitectónico	Cliente: Screen + ViewModel	Separa la vista declarativa (Compose) de la lógica de presentación y el estado.
Inyección de dependencias	Arquitectónico	Cliente (Hilt) y servidor (FastAPI Depends)	Invierte las dependencias hacia abstracciones; favorece el bajo acoplamiento y las pruebas.
Proxy inverso	Arquitectónico	Nginx	Único punto de entrada expuesto a internet: termina el TLS (HTTPS con Let's Encrypt), enruta las peticiones hacia la API interna, oculta la topología del sistema y admite subidas de vídeo grandes (client_max_body_size 200M).
Repository	Estructural	Cliente: SessionRepository (interfaz) + SessionRepositoryImpl	Abstrae el origen de datos; la presentación no conoce la API ni el almacenamiento local.
Data Mapper (ORM)	Estructural	Servidor: modelos SQLAlchemy	Mapea las entidades del dominio a las tablas de la base de datos sin acoplar la lógica de negocio al SQL de MariaDB.
DTO	Estructural	Servidor: esquemas Pydantic (UserResponse, SessionDetail, RepResult...)	Separan el contrato de la API de los modelos ORM, validan los datos de entrada/salida y evitan exponer la estructura interna de la base de datos.
Observer	GoF (comportamiento)	Cliente: Compose observa el estado del ViewModel	La interfaz se recompone de forma reactiva ante los cambios de estado.
Singleton	GoF (creacional)	Servidor: modelos ML (_xgb_frontal, _xgb_lateral, BiLSTM)	Los modelos se cargan una sola vez al importar el módulo, evitando recargar archivos pesados en cada petición HTTP.
Controller	GRASP	routers (servidor) y ViewModel (cliente)	Concentran la recepción y la coordinación de las operaciones del sistema.
Experto en información	GRASP	Servicios readiness y recommendations	Asigna el cálculo a la clase que reúne y conoce los datos necesarios.
Bajo acoplamiento	GRASP	Arquitectura por capas + Repository + DI	Cada módulo tiene una responsabilidad única y depende de abstracciones, no de implementaciones concretas.
Polimorfismo	GRASP	Interfaz SessionRepository	Permite sustituir la implementación (real o de prueba/mock) sin alterar la capa de presentación.

Tabla 14. Patrones de diseño

8 Diseño de interfaces

El diseño de interfaces abarca tanto la interfaz de usuario, con la que interactúa el deportista, como las interfaces de programación que comunican el cliente con el servidor. Este capítulo presenta primero el diseño de la interfaz de usuario: su lenguaje visual, las pantallas principales y el mapa de navegación. Posteriormente, en el apartado 8.2 se presenta el contrato de la API REST.

8.1 Interfaz de usuario

La interfaz se ha construido con Jetpack Compose y el sistema de diseño Material 3. La navegación se gestiona con Navigation Compose y toda la interfaz se presenta en español. La estructura responde a un patrón claro: una pantalla de arranque que decide el destino según el estado de la sesión, un flujo de acceso y registro, y una pantalla principal que actúa como eje desde la que se alcanzan el resto de las funcionalidades. Las pantallas de la aplicación se recogen de forma exhaustiva en el Anexo V (Manual de usuario), por lo que no se reproducen aquí para evitar duplicidades. Este apartado se centra en las decisiones de diseño de la interfaz.

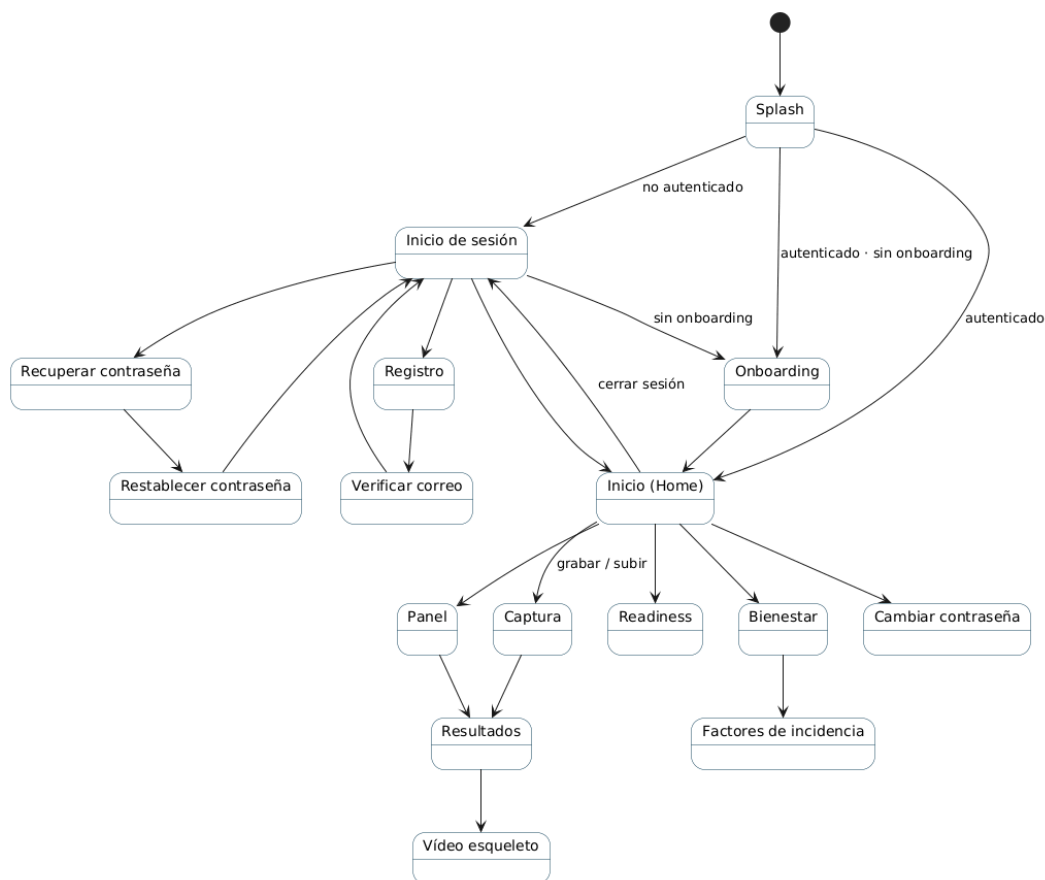


Figura 23. Pantallas de la interfaz

8.2 Interfaces externas de la API

El servidor expone sus servicios mediante una API REST sobre HTTP, versionada bajo el prefijo `/api/v1`. El intercambio de datos se realiza en JSON (salvo la subida de vídeo (multipart)) y la descarga de vídeo e informes (binario). Los contratos de entrada y salida se definen mediante los DTO del apartado 7.1. La autenticación se basa en tokens JWT: salvo los endpoints de acceso (registro, inicio de sesión y recuperación de contraseña), el resto exige el token en la cabecera. Los códigos de estado siguen la semántica estándar. A continuación, se recoge el catálogo de endpoints, agrupado por recurso. Las rutas se muestran a partir del prefijo `/api/v1`.

Autenticación: `/api/v1/auth`

Método	Ruta	Descripción	Acceso	Respuesta
POST	<code>/auth/register</code>	Registrar un usuario	Público	UserResponse (201)
POST	<code>/auth/verify-email</code>	Verificar el correo con el código	Público	MessageResponse
POST	<code>/auth/resent-verification</code>	Reenviar el código de verificación	Público	MessageResponse
POST	<code>/auth/token</code>	Iniciar sesión (obtener <i>tokens</i>)	Público	Token
POST	<code>/auth/refresh</code>	Refrescar el token de acceso	Token de refresco	Token
POST	<code>/auth/forgot-password</code>	Solicitar restablecimiento	Público	MessageResponse
POST	<code>/auth/reset-password</code>	Restablecer la contraseña con código	Público	MessageResponse
PUT	<code>/auth/change-password</code>	Cambiar la contraseña	Autenticado	MessageResponse
POST	<code>/auth/onboarding</code>	Completar el perfil	Autenticado	MessageResponse
GET	<code>/auth/me</code>	Datos del usuario actual	Autenticado	UserResponse
DELETE	<code>/auth/me</code>	Eliminar la cuenta	Autenticado	(204)

Tabla 15. Interfaces API: Autenticación

Sesiones: `/api/v1/sessions`

Método	Ruta	Descripción	Acceso	Respuesta
POST	<code>/sessions/upload</code>	Subir el vídeo y crear la sesión (<i>multipart</i>)	Autenticado	SessionResponse (202)
GET	<code>/sessions/</code>	Listar el historial de sesiones	Autenticado	SessionListResponse[]
GET	<code>/sessions/{id}</code>	Detalle e informe de una sesión	Autenticado	SessionDetail
GET	<code>/sessions/{id}/skeleton-video</code>	Descargar el vídeo con esqueleto (binario)	Autenticado	vídeo
DELETE	<code>/sessions/history</code>	Borrar todo el historial	Autenticado	(204)

Tabla 16. Interfaces API: Sesiones

Analítica: `/api/v1`

Método	Ruta	Descripción	Acceso	Respuesta
GET	/analytics/summary	Resumen estadístico e insights	Autenticado	AnalyticsSummary
GET	/analytics/readiness	Readiness con su desglose	Autenticado	ReadinessBreakdown
GET	/analytics/pdf	Exportar el informe completo en PDF (binario)	Autenticado	PDF

Tabla 17. Interfaces API: Analítica

Check-ins de dolor: /api/v1

Método	Ruta	Descripción	Acceso	Respuesta
POST	/checkins	Registrar un check-in de dolor (EVA)	Autenticado	PainCheckInResponse (201)
GET	/checkins/{session_id}	Check-ins de una sesión	Autenticado	PainCheckInResponse[]

Tabla 18. Interfaces API: Check-in de dolor

Incidencias: /api/v1

Método	Ruta	Descripción	Acceso	Respuesta
GET	/incidents	Listar incidencias	Autenticado	IncidentResponse[]
GET	/incidents/{id}	Detalle de una incidencia	Autenticado	IncidentResponse
POST	/incidents	Crear una incidencia	Autenticado	IncidentResponse (201)
PATCH	/incidents/{id}	Actualizar una incidencia	Autenticado	IncidentResponse
POST	/incidents/{id}/recover	Marcar como recuperada	Autenticado	IncidentResponse
DELETE	/incidents/{id}	Eliminar una incidencia	Autenticado	(204)

Tabla 19. Interfaces API: Incidencias

Bienestar: /api/v1

Método	Ruta	Descripción	Acceso	Respuesta
GET	/wellness/timeline	Línea temporal de bienestar	Autenticado	WellnessTimeline
GET	/wellness/incidents/{id}/factors	Factores previos a una incidencia	Autenticado	IncidentFactors
GET	/wellness/validation	Validación retrospectiva del readiness	Autenticado	ValidationResult

Tabla 20. Interfaces API: Bienestar

Recomendaciones: /api/v1

Método	Ruta	Descripción	Acceso	Respuesta
GET	/recommendations	Recomendaciones activas, por prioridad	Autenticado	RecommendationList

Tabla 21. Interfaces API: Recomendaciones

9 Diseño de datos

El diseño de datos determina cómo se almacena de forma persistente la información del sistema. Se presentan el modelo lógico, que define el esquema relacional con sus tablas, claves y restricciones. Posteriormente se describe cómo se implementa la base de datos.

9.1 Modelo lógico

El modelo lógico define el esquema relacional de la base de datos. El sistema persiste su información en seis relaciones, derivadas de las entidades del modelo de dominio. La figura muestra las tablas, sus columnas con tipo y las relaciones de clave ajena.

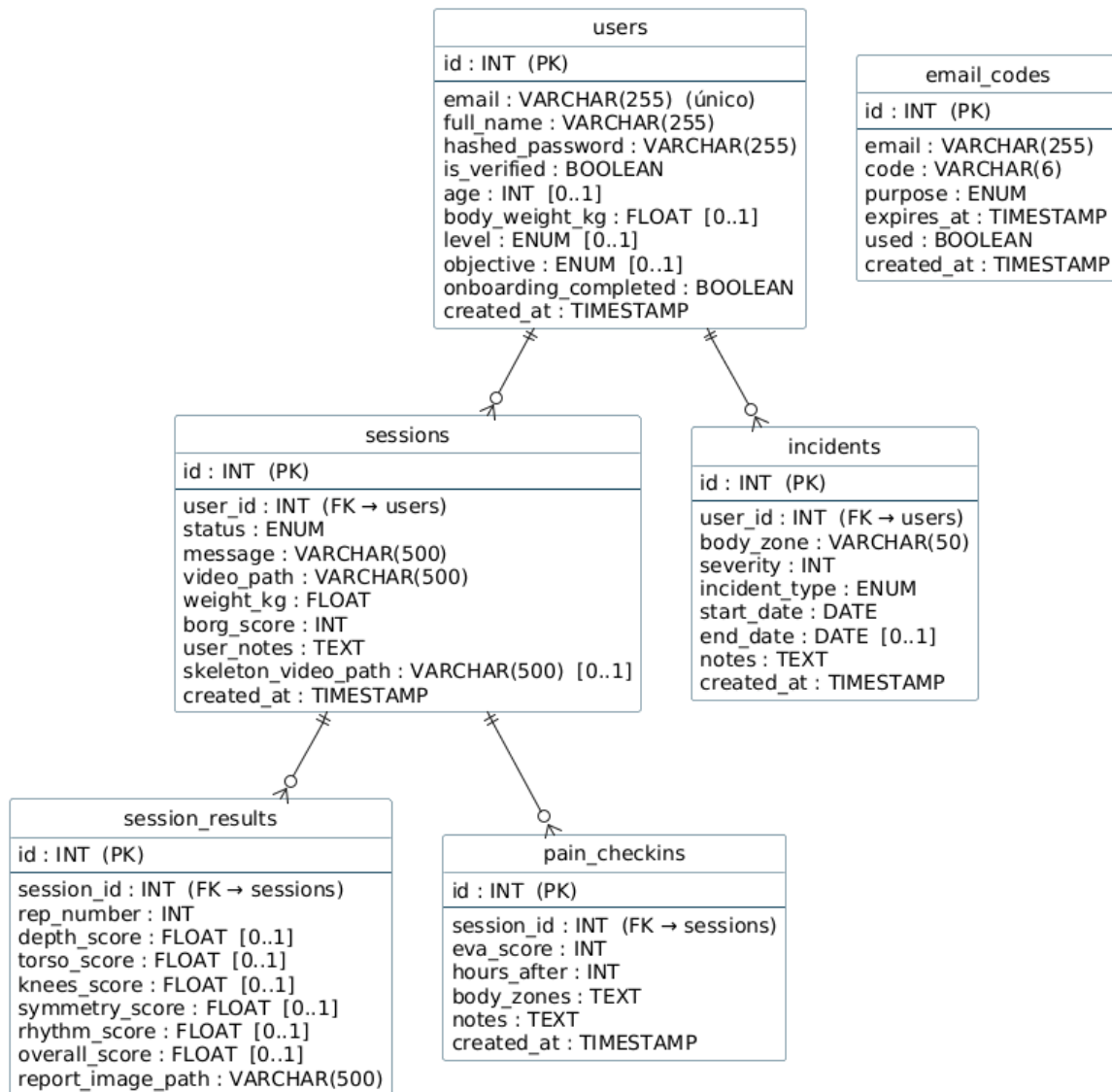


Figura 24. Modelo lógico (BD)

9.1.1 Claves y relaciones

Todas las tablas emplean un identificador entero autoincremental como clave primaria. Las claves ajenas establecen cuatro relaciones uno-a-muchos: un usuario tiene muchas sesiones (`sessions.user_id`) y muchas incidencias (`incidents.user_id`); una sesión tiene muchos resultados por repetición (`session_results.session_id`) y muchos check-ins de dolor (`pain_checkins.session_id`).

9.1.2 Restricciones

El correo del usuario (`users.email`) es único y obligatorio. Los campos del perfil (`age`, `body_weight_kg`, `level`, `objective`) son opcionales, ya que se rellenan en el onboarding. Las puntuaciones de `session_results` admiten valor nulo: en concreto, `knees_score` (valgo) y `symmetry_score` quedan nulas en las grabaciones laterales, donde no son medibles.

9.1.3 Caso particular

`Email_codes`. No tiene clave ajena hacia `users`: se vincula por el correo electrónico, lo que permite generar códigos antes de que la cuenta exista de forma verificada (registro) o cuando el usuario no puede autenticarse (restablecimiento de contraseña).

9.2 Implementación de la base de datos

La persistencia se implementa con el ORM SQLAlchemy sobre MariaDB 11, con el motor InnoDB, que proporciona transacciones e integridad referencial mediante claves ajenas. Cada clase del dominio se mapea a su tabla siguiendo el patrón Data Mapper, de modo que la lógica opera con objetos y es el ORM quien genera el SQL. El esquema no se gestiona con una herramienta de migraciones, sino que se genera a partir de los propios modelos al arrancar la aplicación.

La gestión de las transacciones sigue el patrón Unit of Work, con un alcance distinto según el contexto. En las peticiones HTTP, una dependencia (`get_db`) abre una sesión por petición, la inyecta en el router y la cierra al finalizar. En el procesamiento asíncrono, en cambio, la tarea `process_video` no reutiliza la sesión de la petición, sino que abre su propia sesión: recibe el identificador de la sesión, recarga la entidad y confirma los cambios de forma incremental en cada transición de estado (`encolada` → `procesando` → `completada/fallida`), de modo que el sondeo de la aplicación va observando el avance. La conexión se realiza con el conector PyMySQL sobre un pool con verificación previa de cada conexión.

En el plano físico, además de las claves primarias se definen índices sobre las columnas más consultadas: un índice único sobre el correo del usuario y los índices de las claves ajenas (`user_id`, `session_id`). Por último, una decisión de diseño relevante es que los ficheros pesados, básicamente: los vídeos y las imágenes de informe, no se almacenan en la base de datos, sino en el sistema de archivos del servidor (volúmenes Docker); en las tablas se guarda únicamente su ruta (`video_path`, `skeleton_video_path`, `report_image_path`). Esto mantiene la base de datos ligera y delega el almacenamiento masivo en el sistema de ficheros. La propia base de datos reside, a su vez, en un volumen persistente que conserva los datos entre reinicios del contenedor.

10 Diagrama de despliegue

El diagrama de despliegue describe la topología de ejecución del sistema: los nodos físicos, los artefactos que se despliegan en ellos y las vías de comunicación. MoveInsight se despliega sobre dos nodos: el dispositivo móvil del deportista, donde se instala la aplicación Android, y un único servidor de la Universidad, que aloja el resto del sistema manejado con Docker Compose.

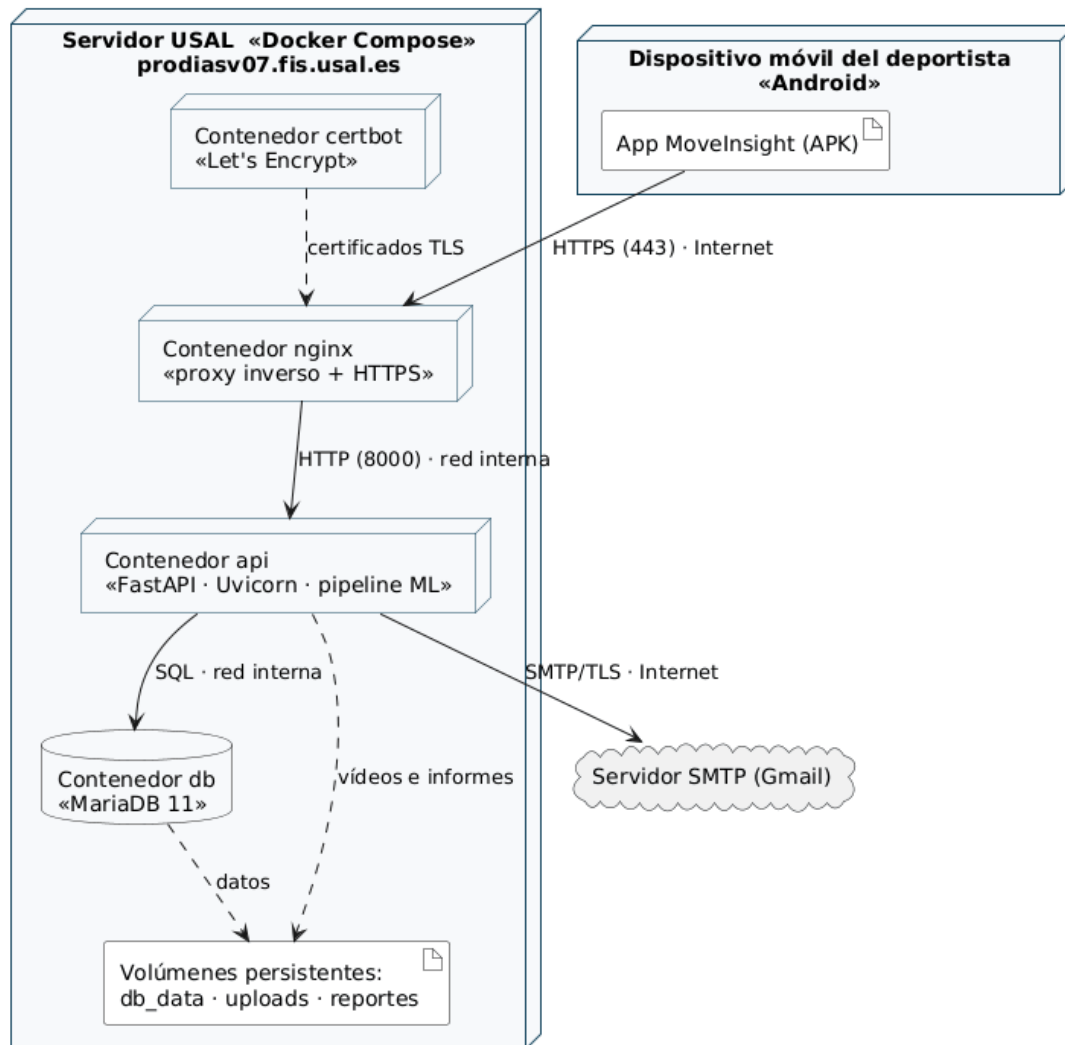


Figura 25. Diagrama de despliegue

En el servidor conviven cuatro contenedores sobre una red interna de Docker. El contenedor nginx es el único expuesto a internet (puertos 80 y 443) y actúa como proxy inverso con terminación TLS, reenviando las peticiones al contenedor api por la red interna. El contenedor api ejecuta la lógica de negocio y la canalización de análisis e incluye los modelos de ML (BiLSTM y XGBoost) y persiste los datos en el contenedor db (MariaDB). El contenedor certbot obtiene y renueva el certificado TLS de Let's

Encrypt. La aplicación móvil se comunica con el servidor exclusivamente por HTTPS, y el servidor envía los correos a través del servidor SMTP externo.

En cuanto a la seguridad y persistencia, que la API y la base de datos no estén expuestas a internet y que el único punto de entrada sea Nginx con HTTPS materializa la decisión arquitectónica ADR-06. La persistencia usa volúmenes Docker con nombre (db_data, uploads, reportes), que conservan tanto la base de datos como los vídeos e informes entre reinicios de los contenedores.

Glosario de términos

Se recogen a continuación los términos técnicos y específicos del dominio empleados a lo largo de este anexo, con el fin de facilitar su comprensión.

- **ACWR (Acute:Chronic Workload Ratio).** Cociente entre la carga de entrenamiento aguda (últimos 7 días) y la crónica (últimos 28 días); indicador de riesgo de sobreentrenamiento cuyo rango óptimo se sitúa entre 0,8 y 1,3.
- **ADR (Architecture Decision Record).** Registro que documenta una decisión arquitectónica: su contexto, la opción adoptada, las alternativas descartadas y sus consecuencias.
- **API REST.** Estilo de interfaz que expone los servicios del servidor mediante operaciones HTTP sobre recursos identificados por URL, con un intercambio de datos sin estado.
- **BiLSTM.** Red neuronal recurrente bidireccional (Bidirectional Long Short-Term Memory); en MoveInsight segmenta las fases de la sentadilla y permite contar las repeticiones.
- **Borg (escala de esfuerzo percibido).** Escala subjetiva con la que el usuario valora la intensidad del esfuerzo de una sesión de entrenamiento.
- **Clean Architecture.** Patrón arquitectónico que organiza el código en capas (presentación, dominio y datos) con las dependencias dirigidas hacia el interior.
- **Cliente-servidor.** Estilo arquitectónico en el que una aplicación cliente consume, a través de la red, los servicios que ofrece un servidor.
- **Docker.** Tecnología que empaqueta una aplicación y sus dependencias en una unidad aislada y reproducible, el contenedor, que se ejecuta de forma uniforme en cualquier máquina.
- **Docker Compose.** Herramienta que orquesta varios contenedores y sus recursos (redes, volúmenes) a partir de un único fichero de configuración.
- **DataStore.** Mecanismo de almacenamiento local de Android empleado para guardar el token de sesión en el dispositivo.
- **DTO (Data Transfer Object).** Objeto que transporta datos entre capas o entre el cliente y el servidor; define el contrato de la API y desacopla la representación externa de los modelos internos.
- **Endpoint.** Punto de acceso concreto de la API, identificado por un método HTTP y una ruta.

- **Estereotipos de análisis.** Clasificación de las clases del modelo de análisis en frontera (interacción con el actor), control (lógica del caso de uso) y entidad (datos del dominio).
- **Estimación de pose.** Técnica de visión por computador que extrae, fotograma a fotograma, la posición de los puntos clave del cuerpo a partir del vídeo.
- **EVA (Escala Visual Analógica).** Escala subjetiva de 0 a 10 con la que el usuario indica el dolor percibido.
- **FastAPI.** Marco de desarrollo web de Python con el que se ha construido la API del servidor.
- **GoF (Gang of Four).** Conjunto clásico de patrones de diseño orientado a objetos, agrupados en creacionales, estructurales y de comportamiento.
- **GRASP.** Conjunto de principios para la asignación de responsabilidades a las clases (Controller, Experto en información, Bajo acoplamiento, entre otros).
- **Hilt.** Biblioteca de inyección de dependencias para Android, que provee y enlaza las dependencias entre las capas del cliente.
- **HTTP / HTTPS.** Protocolo de comunicación de la web; HTTPS es su variante cifrada mediante TLS.
- **InnoDB.** Motor de almacenamiento de MariaDB que aporta transacciones e integridad referencial mediante claves ajenas.
- **Inyección de dependencias.** Patrón por el que una clase recibe sus dependencias desde el exterior en lugar de crearlas, lo que reduce el acoplamiento.
- **Jetpack Compose.** Kit de interfaz de usuario declarativa de Android con el que se construyen las pantallas de la aplicación.
- **JWT (JSON Web Token).** Token firmado que transporta la identidad del usuario y permite autenticar las peticiones sin mantener estado en el servidor.
- **KPI (Key Performance Indicator).** Indicador clave de rendimiento; en MoveInsight, cada métrica biomecánica por repetición: profundidad, torso, valgo, simetría y ritmo.
- **Let's Encrypt.** Autoridad de certificación gratuita que emite los certificados TLS empleados para habilitar HTTPS.
- **MariaDB.** Sistema gestor de base de datos relacional empleado para la persistencia de la información.
- **MediaPipe.** Biblioteca de visión por computador empleada para la estimación de la pose corporal.

- **Modelo C4.** Notación que describe la arquitectura del software en niveles de detalle crecientes: contexto, contenedores, componentes y código.
- **MVVM (Model-View-ViewModel).** Patrón arquitectónico que separa la vista de la lógica de presentación mediante un ViewModel que gestiona el estado.
- **Nginx (proxy inverso).** Servidor que recibe las peticiones externas y las reenvía a los servicios internos; aquí actúa como proxy inverso y termina las conexiones HTTPS.
- **ORM (Object-Relational Mapping).** Técnica que traduce las clases de objetos a tablas relacionales, permitiendo operar con objetos en lugar de con sentencias SQL.
- **Pool de conexiones.** Conjunto de conexiones a la base de datos reutilizables, que evita abrir y cerrar una conexión en cada operación.
- **Procesamiento asíncrono.** Modelo en el que una operación costosa se ejecuta en segundo plano sin bloquear la respuesta al usuario.
- **Pydantic.** Biblioteca de Python para validar y serializar datos; define los esquemas (DTO) de la API.
- **Readiness.** Índice de disposición para el entrenamiento (0–100, representado con un semáforo) que resume el estado de forma combinando fatiga, carga, dolor y técnica.
- **Repository (patrón).** Patrón que abstrae el acceso a los datos tras una interfaz, de modo que el resto del código ignora el origen concreto de la información.
- **Retrofit.** Biblioteca de Android para consumir APIs REST de forma declarativa.
- **Sección de Análisis.** Sección de la aplicación que agrega los indicadores de progreso del deportista: récords, comparativa con la sesión anterior y resumen semanal.
- **Sentadilla.** Ejercicio de fuerza de tren inferior; es el único ejercicio que analiza el sistema.
- **SMTP (Simple Mail Transfer Protocol).** Protocolo estándar para el envío de correo electrónico; el servidor lo utiliza para los correos de verificación y restablecimiento de contraseña.
- **Sondeo (polling).** Técnica por la que el cliente consulta periódicamente al servidor el estado de una operación hasta que esta finaliza.
- **SQLAlchemy.** Biblioteca ORM de Python empleada para el mapeo objeto-relacional y el acceso a la base de datos.

- **TLS (Transport Layer Security).** Protocolo criptográfico que cifra la comunicación en red; es la base de HTTPS.
- **UML (Unified Modeling Language).** Lenguaje gráfico estándar para modelar sistemas de software, empleado en todos los diagramas de este anexo.
- **Unit of Work.** Patrón que agrupa un conjunto de operaciones sobre la base de datos en una única transacción coherente.
- **Uvicorn.** Servidor de aplicaciones ASGI sobre el que se ejecuta FastAPI.
- **Valgo de rodilla.** Desviación de la rodilla hacia el interior durante el movimiento; es uno de los KPIs biomecánicos evaluados.
- **Volumen (Docker).** Mecanismo de almacenamiento persistente de Docker que conserva los datos aunque el contenedor se reinicie o se recree.
- **Wireframe / mockup.** Boceto o maqueta de una pantalla que representa su estructura y disposición antes de la implementación.
- **XGBoost.** Algoritmo de aprendizaje automático basado en árboles de decisión potenciados; clasifica los KPIs de cada repetición.